A Survey on Edge Performance Benchmarking

Varghese, B., Wang, N., Bermbach, D., Hong, C.-H., de Lara, E., Shi, W., & Stewart, C. (2021). A Survey on Edge Performance Benchmarking. *ACM Computing Surveys*, *54*(3), Article 66. https://doi.org/10.1145/3444692

Published in:
ACM Computing Surveys

Document Version:
Peer reviewed version

Queen's University Belfast - Research Portal:
Link to publication record in Queen's University Belfast Research Portal

Publisher rights
Copyright 2020 ACM
This work is made available online in accordance with the publisher's policies. Please refer to any applicable terms of use of the publisher

General rights
Copyright for the publications made accessible via the Queen's University Belfast Research Portal is retained by the author(s) and / or other copyright owners and it is a condition of accessing these publications that users recognise and abide by the legal requirements associated with these rights.

Take down policy
The Research Portal is Queen's institutional repository that provides access to Queen's research output. Every effort has been made to ensure that content in the Research Portal does not infringe any person's rights, or applicable UK laws. If you discover content in the Research Portal that you believe breaches copyright or violates any law, please contact openaccess@qub.ac.uk.

Open Access
This research has been made openly available by Queen's academics and its Open Research team. We would love to hear how access to this research benefits you. – Share your feedback with us: http://go.qub.ac.uk/oa-feedback

Download date:07. Jun. 2026

A Survey on Edge Performance Benchmarking

Blesson Varghese[a], Nan Wang[b], David Bermbach[c], Cheol-Ho Hong[d], Eyal de Lara[e], Weisong Shi[f], and Christopher Stewart[g]

[a]Queen's University Belfast, UK; [b]Durham University, UK; [c]TU Berlin, Germany, and Einstein Center Digital Future, Germany; [d]Chung-Ang University, S. Korea; [e]University of Toronto, Canada; [f]Wayne State University, USA; [g]Ohio State University, USA

This article may be cited as: ***B. Varghese, N. Wang, D. Bermbach, C. -H. Hong, E. de Lara, W. Shi and C. Stewart, "A Survey on Edge Performance Benchmarking," accepted to ACM Computing Surveys, 2020.***

Edge computing is the next Internet frontier that will leverage computing resources located near users, sensors, and data stores to provide more responsive services. Therefore, it is envisioned that a large-scale, geographically dispersed, and resource-rich distributed system will emerge and play a key role in the future Internet. However, given the loosely coupled nature of such complex systems, their operational conditions are expected to change significantly over time. In this context, the performance characteristics of such systems will need to be captured rapidly, which is referred to as performance benchmarking, for application deployment, resource orchestration, and adaptive decision-making. *Edge performance benchmarking* is a nascent research avenue that has started gaining momentum over the past five years. This article first reviews articles published over the past three decades to trace the history of performance benchmarking from tightly coupled to loosely coupled systems. It then systematically classifies previous research to identify the system under test, techniques analyzed, and benchmark runtime in edge performance benchmarking.

Edge computing | Edge performance benchmarking | System under test | Quality | Benchmark runtime

1. Introduction

The computing landscape has changed significantly over the past three decades. Loosely coupled and geographically dispersed systems have begun replacing tightly coupled monolithic systems (1). One example is from two decades ago, when computing resources that were distributed across numerous organizations and continents were connected under the umbrella of grid computing. Grids offered the unique capability of processing large datasets near data sources without requiring the transfer of data from a distributed workflow to a central system (2). Subsequently, computing became a utility offered remotely through the cloud (3).

Although the cloud is the main computing model adopted for many Internet-based applications, it has been recognized as an untenable model for the future. This is because billions of devices and sensors are connected to the Internet, and the data generated by these sources cannot be transferred and processed in geographically distant cloud data centers without incurring considerable communication delays (4). Therefore, the next disruption in the computing landscape is to distribute infrastructure resources and application services further, to bring computing closer to the edge of the network and data sources (5, 6). In this article, we use the term "edge computing" to refer to the use of resources located at the edge of a network, such as routers and gateways or dedicated micro data centers, to either provide applications with acceleration by co-hosting services in cooperation with the cloud or by hosting them natively or entirely on edge resources (7).

The inclusion of edge resources for computing creates a large-scale, geographically dispersed, resource-rich distributed system that spans multiple technological domains and ownership boundaries. Such a complex system will be transient, meaning that resources, their availability, and characteristics will change over time. For example, an edge resource previously available for an application may become unavailable based on a recent fault or because the operating system of a target resource may change during maintenance (8, 9). In this context, it is essential to address the challenge of understanding the relative performance of applications by comparing diverse target hardware platforms from different vendors and their impact on performance when system software changes or new networking protocols are introduced (10). This has motivated the development of ***edge performance benchmarking***[1] (11, 12).

Performance benchmarking is the process of inducing stress on a system while closely observing its responses using a wide range of quality metrics. Typically, synthetic or application-driven workloads are executed on a system under test, such as a virtual machine, a storage system, a stream processing system, or a specific application component, while measuring quality characteristics, such as I/O throughput, end-to-end communication, or computation latency. Unlike alternative approaches such as predictive methods or simulation, insights into real system behaviors can be obtained by replicating the conditions of a production environment (13).

To the best of our knowledge to date, a survey on edge performance benchmarking is not available in the literature. Hence, this article focuses on the following aspects: (i) Tracing the history of the development of performance benchmarking over the past three decades for high-performance computing (HPC), grid, and cloud systems, (ii) cataloging and examining different edge performance benchmarks, and (iii) reviewing the system under test, techniques analyzed, and benchmark runtime that facilitate edge performance benchmarking.

Figure 1 presents a histogram of the total number of research publications reviewed in this article from 1976 and 2020 in the following categories: (i) books and book chapters; (ii) reports, including preprint articles or white papers and doctoral research theses; (iii) conference or workshop papers; and (iv) journal or magazine articles. More than 83% of the articles reviewed were published after 2010 and more than 61% of the articles were published after 2015.

A. Survey Method. The survey method adopted for preparing this article is based on an approach presented in a previous survey article (14). This method includes (1) defining the objectives of the survey, (2) defining research questions, (3)

[1]This article will use the terms "performance benchmarking" and "benchmarking" interchangeably. However, the focus of this article is on edge performance benchmarking.

[a]Corresponding e-mail: b.varghese@qub.ac.uk

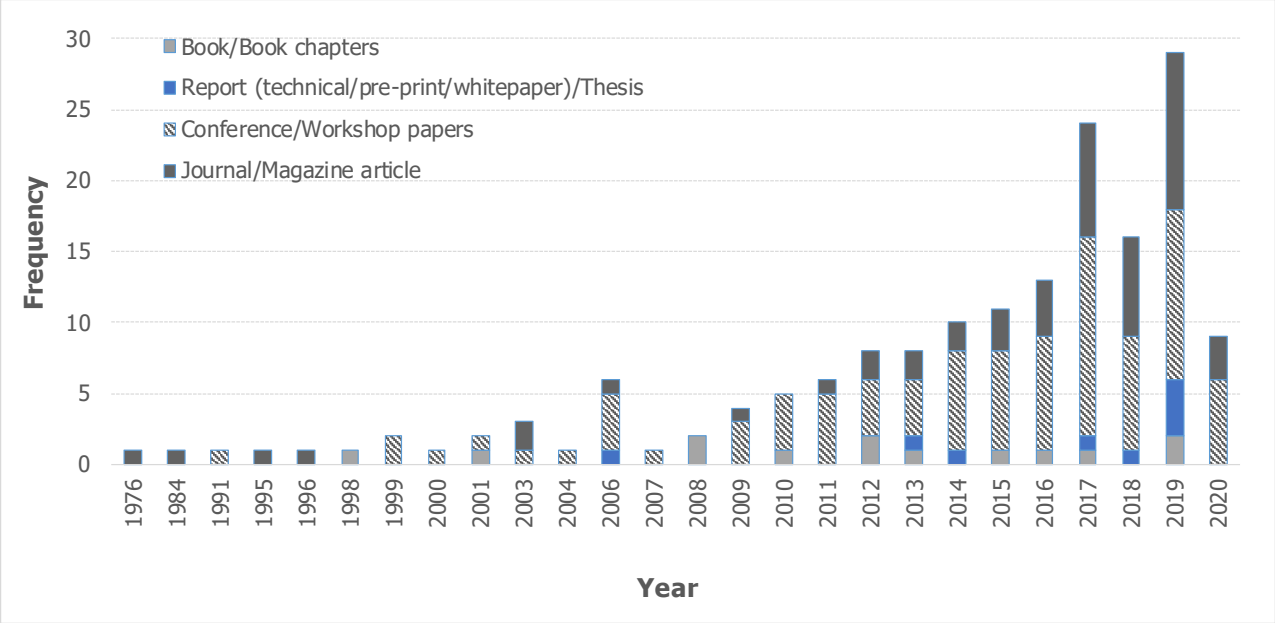

Fig. 1. Histogram of the research publications reviewed in this article.

selecting keywords for searching, and (4) identifying criteria for including or excluding research. These aspects are defined below.

(1) The *objectives* of this survey are defined as follows:

O1 Provide the research community with a catalog of research related to edge performance benchmarking.

O2 Trace the development timeline of performance benchmarking for edge systems.

O3 Understand the key dimensions of existing research on edge performance benchmarking.

O4 Discuss directions for future research to extend the application of edge performance benchmarking.

(2) The *research questions* addressed by this survey are defined as follows:

RQ1 To which systems is edge performance benchmarking applied? This will be discussed in Section 4.

RQ2 Which techniques are analyzed by edge performance benchmarking? This will be discussed in Section 5.

RQ3 What are the runtime environments that edge performance benchmarks operate in? This will be discussed in Section 6.

(3) Publication platforms such as the ACM Digital Library, IEEEXplore, ScienceDirect, arXiv, and Google Scholar were considered. The primary *keywords* were a combination of edge (fog, mobile edge, cloud-edge, cloudlet) and benchmarking (benchmark, benchmark suite, micro benchmark, and macro benchmark) with additional keywords such as performance, system, and distributed systems.

(4) The resulting works were screened to identify the most relevant works. A total of 3,764 publications were considered and screened down to 689 publications. The initial filters applied were based on the title, followed by the relevance of the abstract. Two types of performance benchmarking works are available, namely explicit and implicit edge performance benchmarking. For research to be selected as explicit performance benchmarking, a benchmarking method, specific benchmark, or toolchain for facilitating performance benchmarking had to be presented, which was determined by inspecting complete papers. Such papers generally contribute to the field of edge benchmarking. We selected 21 works containing explicit performance benchmarks, which are cataloged in Section 3.

Implicit performance edge benchmarking works were also included if they presented an evaluation of the performance of a system under testing, technique developed, or runtime, even though such works did not explicitly highlight a benchmark, benchmarking methodology, or benchmarking suite. Studies that did not present a comparative analysis of the system under test, technique developed, or runtime were not considered for implicit benchmarking. The selected works often use novel workloads and measurement approaches that could potentially be incorporated into explicit edge benchmarking research. A total of 99 implicit performance edge benchmarking publications were selected. Selections based on the above mentioned criteria were validated by at least two of the authors of this article.

This article considers a wide range of papers and Internet sources related to edge performance benchmarking. For this purpose, we focus on explicit and implicit edge performance benchmarks. We do not claim completeness for our selected set of implicit benchmarks for the following two reasons. First, the body of work on edge computing (including closely related topics such as fog computing or mobile edge computing) is very large and cannot be considered within a single survey paper. Second, explicit edge benchmarks can be identified objectively, whereas implicit benchmarks are subjective. Based on careful analysis and validation by at least two authors, the set of selected implicit benchmarks may not necessarily be complete, but it contains no false positives. Therefore, the subset of implicit benchmarks considered within the scope of this article

adds value to this survey and to the emerging field of edge performance benchmarking.

The remainder of this article is organized as follows. Section 2 provides a brief history of performance benchmarking. Section 3 catalogs different edge performance benchmarks. Section 4 presents a review of systems under testing in edge performance benchmarking. Section 5 reviews the techniques analyzed in edge performance benchmarking. Section 6 surveys runtime execution environments and deployments in edge performance benchmarks. Section 7 discusses future directions for additional research and concludes this paper.

2. A Brief Timeline of Performance Benchmarking

Performance benchmarks have played an important role in the evolution of the computing landscape and represent an important field of research and development. CPU or processor-related benchmarks have existed since the 1970s. For example, the Whetstone benchmark was developed to measure the floating-point arithmetic performance (15). Dhrystone is another benchmark developed in the 1980s to evaluate the performance of integers (16).

This section provides a brief history of the development of the field of benchmarking of distributed systems over the past three decades, as shown in Figure 2. Specifically, both tightly coupled HPC clusters and supercomputers (highlighted in green), as well as more loosely coupled infrastructure, such as the grid (highlighted in bronze) and the cloud (highlighted in turquoise), are considered. The next section will consider edge computing benchmarks (highlighted in red color), which are the main focus of this article.

This article does not present an exhaustive timeline of performance benchmarking within computer science, but highlights some of the key milestones that have shaped performance benchmarking research for current distributed systems and more specifically edge computing, which is discussed in Section 3. In general, the observed pattern is that post-1990 HPC benchmarking, post-2000 grid benchmarking, post-2010 cloud benchmarking, and post-2015 edge benchmarking achieved major milestones. This trend seems to suggest that benchmarks for more loosely coupled distributed systems are gaining prominence as a result of the decentralization and distribution of resources within the computing landscape.

A. HPC benchmarking. In 1979, the LINPACK benchmark was developed. This benchmark eventually evolved into the High-Performance LINPACK (HPL) benchmark (17). This benchmark is computationally intensive and measures the floating point rate of execution by solving a dense system of linear equations. HPL is a de facto benchmark used for capturing the performance of supercomputers and clusters in the TOP500[2] list launched in 1993.

The NAS Parallel Benchmarks (NPB) were launched in 1991 and are based on computational fluid dynamics (CFD) applications (18). The Standard Performance Evaluation Corporation (SPEC) launched several HPC benchmarks in 1996 (19). They focused on three industrial applications, namely seismic processing, computational chemistry, and climate modeling.

More than a decade after the TOP500 project was launched, energy efficiency became an important metric that influenced the development of parallel computing systems (20). This resulted in the launch of the Green500[3] project in 2005, which evaluated floating point rates of execution in the context of power consumption. A methodology for measuring and reporting the power used by an HPC system was developed[4]. In 2016, Green500 was integrated with the TOP500 project.

The HPC Challenge benchmark was launched in 2005 to measure performance and productivity (21). This benchmark includes the STREAM benchmark for measuring sustainable memory bandwidth (22).

The field commonly known as big data became popular in the context of HPC clusters, and hence the GridMix[5] benchmark for Hadoop clusters emerged in 2007.

The High Performance Conjugate Gradient (HPCG) is a relatively new benchmark that was launched in 2013 to rank supercomputers and clusters in a more balanced manner (23). The performance of this benchmark is influenced by memory bandwidth.

In 2016, several benchmarks utilized on the UK's national supercomputer ARCHER were presented[6]. These benchmarks are a combination of real applications (DFT, molecular mechanics-based, CFD, and climate modeling) developed by the UK Met Office and synthetic benchmarks from the HPC Challenge.

Because HPC has become more heterogeneous with the advent of hardware accelerators, such as GPUs, novel benchmarks have begun to emerge. These benchmarks include the RODINIA (24) and SHOC GPU benchmarks (25), as well as the more recent Mirovia benchmarks (26).

B. Grid Benchmarking. As academic and research organizations have begun connecting clusters of computers, geographically dispersed and heterogeneous grids have become popular for scientific computing. Key metrics that are relevant to grid benchmarking included turnaround time and throughput because data originated from different geographic locations in a scientific workflow is executed on grids (27).

One of the first benchmarks for evaluating computing performance on grids was the GridNPB, which was released in 2001. This benchmark is an NPB distribution that uses the Globus grid middleware (28).

In 2003, GridBench, which allows benchmarks to be specified using a definition language that is compiled into specification languages supported by grid middleware, was released (29).

In 2004, the DiPerF benchmarking tool was used to generate performance statistics for grids to predict grid performance (30).

GRENCHMARK was developed for generating synthetic workloads for benchmarking grids (31). This approach was adopted in ServMark for automating benchmarking pipelines (32).

Additional benchmarks such as GrapBench have provided flexibility to benchmarking approaches by considering variations in problems and machine sizes of applications and the grid, respectively (33). Furthermore, CROWNBench generates synthetic workloads for benchmarking (34). Domain-specific

[2] https://www.TOP500.org/

[3] https://www.TOP500.org/green500/

[4] https://www.TOP500.org/static/media/uploads/methodology-2.0rc1.pdf

[5] https://hadoop.apache.org/docs/r1.2.1/gridmix.html

[6] https://www.archer.ac.uk/documentation/white-papers/benchmarks/UK_National_HPC_Benchmarks.pdf

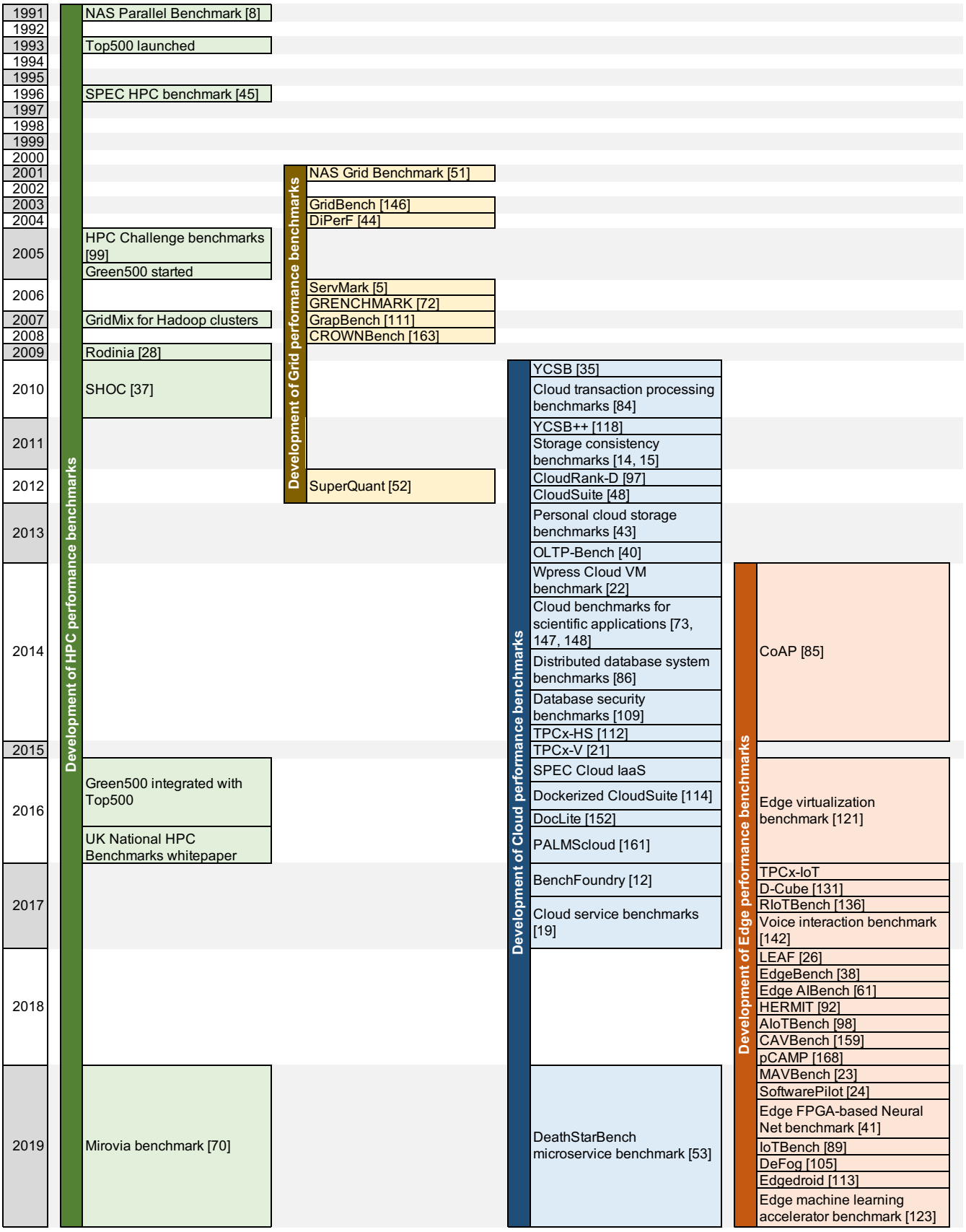

Fig. 2. Brief history of the development of performance benchmarking for HPC, grid, cloud and edge systems.

benchmarks, such as those relevant to financial workloads, have also been developed (35).

C. Cloud Benchmarking. Benchmarking or web-based systems, including open, closed, and partially closed systems, have also been considered (36). One of the earliest benchmarks for web servers was SPECweb96[7]. In 2010, the first cloud-specific benchmarks were released, namely the Yahoo! Cloud Service Benchmark (YCSB) (37) and transaction processing benchmarks (38). YCSB++ is an extension of the YCSB that benchmarks scalable column stores (39). Various approaches for benchmarking scientific applications in the cloud (40–42) and virtual machines (VMs) (43) have been proposed. Moreover, the aspect of fairness in benchmarking has been considered (44). Several benchmarks for the consistency of cloud storage services have been developed (45, 46).

A number of cloud performance benchmarks were developed between 2012 and 2015. Two benchmarks developed by the Transaction Processing Performance Council (TPC) are particularly noteworthy. The first is the TPCx-V (47) benchmark, which is a virtual machine benchmark for database workloads. The second is the TPCx-HS (48) benchmark, which is a big data benchmark on the cloud. Other benchmarks include CloudRank-D (49) and CloudSuite (50) for resources and services. Benchmarks for cloud storage (51, 52), relational databases (53), database security (54), and the scalability and elasticity of distributed databases have also been developed (55).

The SPEC benchmarks for infrastructure-as-a-service clouds were launched in 2016 (SPEC Cloud IaaS 2016[8]). At this time, container-based benchmarking suites such as DocLite (56) and a containerized version of CloudSuite (57) also emerged. Moreover, novel cloud hardware architectures could be evaluated using the PALMScloud benchmark suite (58), and a framework for benchmarking cloud storage services was introduced (59). Additionally, a comparison of various benchmarking suites for measuring the quality of cloud services from a client perspective was introduced (13). More recently, this has led to the development of benchmarks for microservices on distributed clouds (60, 61) and benchmarks that can be used in continuous integration processes (62, 63).

3. Edge Performance Benchmarking

This section highlights developments in edge performance benchmarking and then defines a classification that is used in this article for presenting the different dimensions of research undertaken in the context of edge performance benchmarking.

As shown in Figure 2, edge performance benchmarking has been under development since 2015. These benchmarks cover a wide range of resources, including (i) end devices, such as IoT sensors, smartphones, and user gadgets, including wearables; (ii) computational resources located at the edge of a wired network, including routers, switches, gateways, and dedicated resources, including embedded computers and micro clouds; and (iii) cloud resources. Different benchmarks focus on capturing the performance of these resources in both isolated and networked execution contexts.

Existing research on edge performance benchmarking can be classified into the following three dimensions, as shown in Figure 3:

- *System under test* refers to the hardware infrastructure and software platforms that are benchmarked. This aspect is detailed in Section 4 and aims to address **RQ1**, which was posed in Section A.

- *Techniques analyzed* refers to the application optimization options, resource allocation, and application offloading and deployment options analyzed by edge benchmarks, which are discussed in Section 5, where we aim to address **RQ2**, which was posed in Section A.

- *Benchmark runtime* refers to the software and data characteristics of an execution environment and deployment destination, including test beds, which are considered in Section 6, where we aim to address **RQ3**, which was posed in Section A.

As highlighted in Section A, two types of edge performance benchmarking research can be observed in the literature. The first is explicit performance benchmarking, which is defined as research on developing a benchmarking method, benchmark, or toolchain to facilitate performance benchmarking. The second is implicit performance benchmarking, which refers to research that presents evaluations to capture and compare the performances of any of the aforementioned dimensions (i.e., system under test, techniques analyzed, and benchmark runtime) without specifically presenting a method, benchmark, or toolchain.

Table 1 summarizes existing explicit performance benchmarking research that is relevant to edge computing systems by considering the benchmark type (micro/macro), application domain, benchmarks used, and destination platforms (device, edge, and cloud) of 21 edge benchmarking techniques or suites. Micro benchmarks refer to benchmarks that capture system-level (CPU, memory, network, storage) performance metrics. By contrast, macro benchmarks refer to benchmarks that capture application-specific system performance metrics for different application domains. Only two benchmarks capture both micro and macro benchmarks, namely RIoTBench (64) and AIoTBench (65). The majority of benchmarks are macro benchmarks and only two utilize generic workloads, namely EdgeBench (66) and DeFog (10). Moreover, only five benchmarks can capture the performance of a complete computation pipeline consisting of a device, edge, and cloud. This can be attributed to the lack of readily available large-scale test beds (although a few are available) for experimentation that integrate a cloud and edge for end users.

The large body of research on edge benchmarking considered in this article relies on either trace data obtained from simulators or on simulators themselves for evaluation, which is contrary to the classic definition of benchmarking. We anticipate experimental benchmarking approaches to be adopted as edge computing matures as a research area and more realistic test beds become readily available.

The next three sections will consider both explicit and implicit edge performance benchmarking research and examine this research across the dimensions of system under test, techniques analyzed, and benchmark runtime. Each subsection is organized to discuss explicit edge performance benchmarks first, followed by implicit edge performance benchmarks.

[7] https://www.spec.org/web96/

[8] https://www.spec.org/cloud_iaas2016/

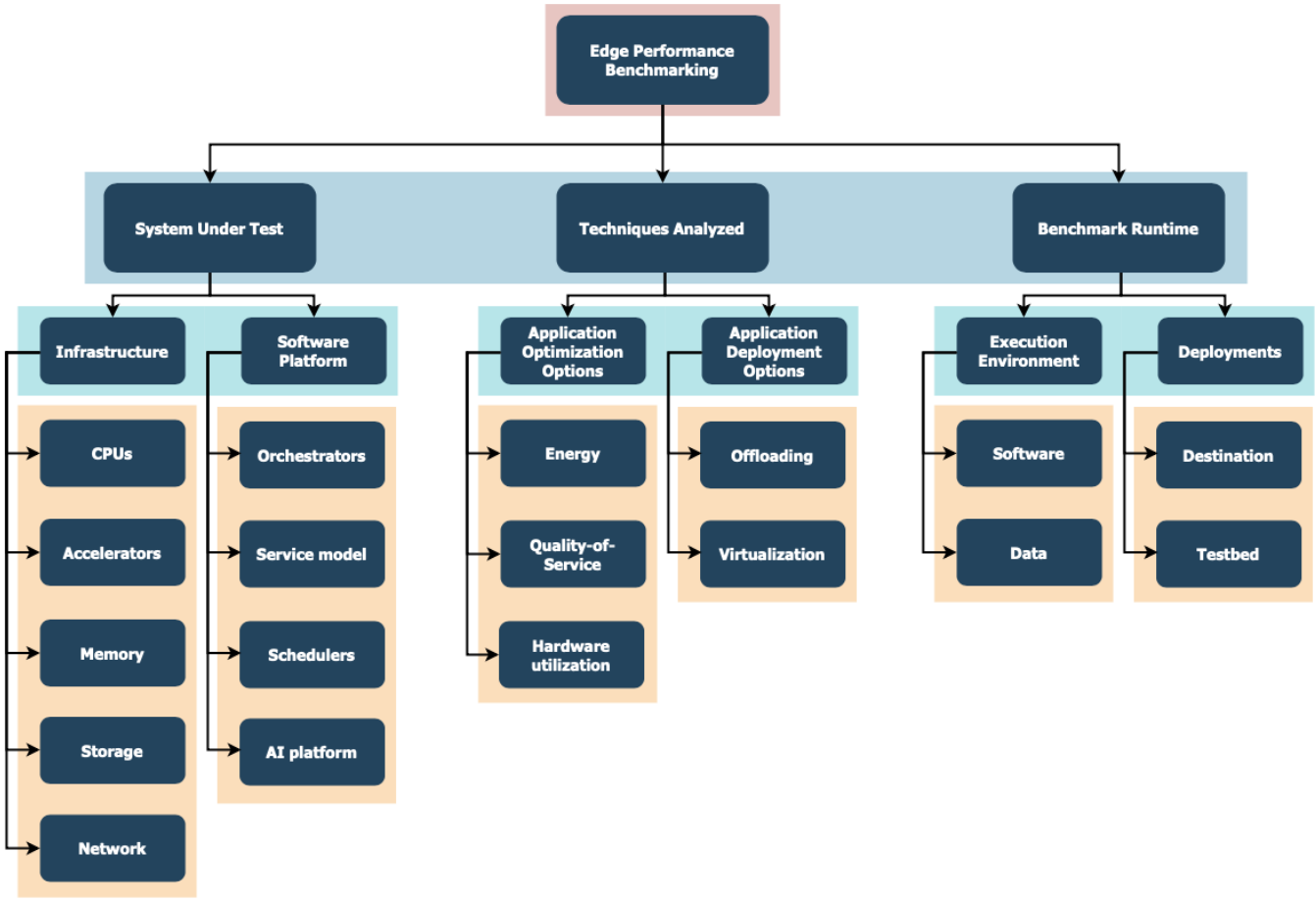

Fig. 3. Classification of edge performance benchmarking under three dimensions: system under test, techniques analyzed, and benchmark runtime.

Table 1. List of relevant Edge computing benchmarks, including their type (micro/macro), application domain, benchmarks used, and destination (D - device, E - edge and C - cloud).

Year	Suite/technique	Type	Domain	Benchmarks used	Destination
2014	CoAP benchmark (67)	Micro	-	lmbench	D
2016	Virt. benchmark (68)	Micro	-	NBench, Sysbench, HPL, bonnie++, DD, Stream, Netperf	D
2017	Virt. benchmark (69)	Micro	-	Sysbench, mbw, fio, iperf, dstat	D
	Voice benchmark (70)	Macro	Voice	Mycroft platform	D, E, C
	RIoTBench (64)	Micro	Stream processing	27 IoT steam processing tasks	D, C
		Macro		Transform and load, Statistical summarization, Predictive analytics	
	TPCx-IoT[9]	Macro	Power grid	Data ingestion and concurrent queries (based on YCSB)	D, E, C
	D-Cube (71)	Micro	Low power systems	6 wireless protocols	D
2018	CAVBench (72)	Macro	Autonomous vehicles	SLAM, Object tracking, Battery diagnostics, Speech recognition, Video analytics	E
	EdgeBench (66)	Macro	Generic	Speech-to-text, Image recognition, Scalar sensor	E, C
	Edge AIBench (73)	Macro	Machine learning	Patient monitor, Surveillance camera, Speech/facial and road sign recognition	D, E, C
	HERMIT (74)	Macro	Internet of Medical Things	Physical activity estimation, Advanced encryption standard, Sleep apnea detection, heart rate variability, Histogram equalization, Inverse radon transform, K-means clustering, Lempel-Ziv-Welch compression, Blood pressure monitor	D
	LEAF (75)	Macro	Federated learning	Image classification, Sentiment analysis, Text prediction	E
	AIoTBench (65)	Micro	Machine learning on mobile devices	Neural network layers - convolution, pointwise convolution, depthwise convolution, matrix multiply, pointwise add, ReLU/sigmoid activation, max/avg. pooling	D
		Macro		Image and Speech recognition, Language translation	
	pCAMP (76)	Macro	Machine learning	TensorFlow, Caffe2, MXNet, PyTorch, TensorFlow Lite	D, E, C
2019	DeFog (10)	Macro	Generic	Object classification, Speech-to-text, Text-audio alignment, Geo-location based mobile game, IoT edge gateway application, Real-time face detection	D, E, C
	Edgedroid (77)	Macro	Human-in-the-loop applications	Cognitive assistance application for assembling LEGO	D, E
	MAVBench (78)	Macro	Micro aerial vehicles	Scanning, Aerial photography, Package delivery, 3D mapping, Search and rescue	D (Drone), C
	IoTBench (79)	Macro	Vision and speech	Video summarization, Stereo image matching, Image recognition, Scan matching, Voice feature extraction, Signals enhancement, Data compression	D
	SoftwarePilot[10] (80)	Macro	Unmanned aerial vehicle	Aerial photography, Crop surveillance, Rescue and discovery, Facial recognition	E, C
	Machine learning accelerator benchmark (81)	Macro	Low power machine learning accelerators	Mobilenet on TensorFlow and OpenVINO	D, E, C
	Edge FPGA-based Neural Net benchmark (82)	Macro	Neural network	Keyword spotting application	E

System under test characteristics considered by edge performance benchmarks	Infrastructure					Software platforms			
	CPUs	Accelerators	Memory	Storage	Network	Orchestrators	Service Model	Schedulers	AI Platforms
CoAP benchmark (67)	Y	N	Y	N	N	N	N	N	N
Virt. benchmark (68)	Y	N	Y	Y	Y	N	N	N	N
Virt. benchmark (69)	Y	N	Y	Y	N	N	N	N	N
Voice benchmark (70)	Y	N	N	N	N	N	N	N	N
RIoTBench (64)	Y	N	Y	N	N	N	N	N	N
D-Cube (71)	Y	N	N	N	N	N	N	N	N
CAVBench (72)	Y	N	Y	N	N	N	N	N	N
EdgeBench (66)	Y	N	Y	N	N	N	Y	N	N
Edge AIBench (73)	N	N	N	N	N	N	N	N	Y
HERMIT (74)	Y	N	Y	N	N	N	N	N	N
LEAF (75)	N	N	N	N	N	N	N	N	Y
AIoTBench (65)	N	N	N	N	N	N	N	N	Y
pCAMP (76)	Y	Y	N	N	N	N	N	N	N
DeFog (10)	Y	N	N	N	Y	N	N	N	N
Edgedroid (77)	N	N	N	N	N	Y	N	N	N
IoTBench (79)	Y	N	N	N	N	N	N	N	N
Machine learning accelerator benchmark (81)	Y	Y	N	N	N	N	N	N	N
Edge FPGA-based Neural Net benchmark (82)	N	Y	N	N	N	N	N	N	N

Table 2. Comparison of the characteristics of system under test in existing edge performance benchmarks

4. System Under Test

In this section, the systems under testing in edge performance benchmarking are examined by considering two components, infrastructure and software platforms, which are discussed in the following subsections. Each subsection will discuss both explicit and implicit edge performance benchmarks. The explicit performance benchmarks are listed in Table 2.

A. Infrastructure. Embedded CPUs, accelerators, memory, storage hardware, and the network are the infrastructure resources that are considered in edge performance benchmarking.

A.1. CPUs. As shown in Table 2, the majority of edge performance benchmarks can measure the performance of CPUs used on the edge (five of these benchmarks cannot). Many edge benchmarks are evaluated on single-board computers (SBCs), such as Raspberry Pi (11), acting as edge resources. An SBC is a circuit board consisting of a CPU, memory, network, storage, and other components. Most SBCs adopt ARM processors as CPUs and have low cost and low power characteristics. Additionally, the performance of modern ARM processors is comparable to that of other general-purpose CPUs (83).

The CoAP benchmark (67) utilizes lmbench[11] for benchmarking ARM processors in Raspberry Pi, BeagleBone, and BeagleBone Black SBCs in terms of bandwidth and latency. The processing overhead of key operations in a modern WSN gateway can also be measured. An ARM Cortex-A7 dual-core processor hosted by Cubieboard2 was benchmarked (68) using NBench[12], sysbench[13], and the High-Performance Linpack (HPL) benchmark (84). In a recent study, a wide range of ARM-based SBCs, including the Raspberry Pi 2 model B (RPi2), Raspberry Pi 3 model B (RPi3), Odroid C1+ (OC1+), Odroid C2 (OC2), and Odroid XU4 (OXU4) (69), were benchmarked by using sysbench to stress their CPUs. In terms of CPU performance, the Odroid C2 with a Quad-core 2 GHz ARM v8 Cortex-A53 outperformed the other tested SBCs. In terms of power consumption, the Raspberry Pi boards, OC1+, and OC2 achieved high energy efficiency, whereas the OXU4 consumed three to seven times more power than the other SBCs.

An ARM Cortex A53 CPU hosted by RPi3 was benchmarked (70) using Mycroft[14], which is an open-source voice assistant. The latency of the pipeline stages of voice interaction was measured, and the results indicated that cloud services outperform RPi, meaning RPi is more suitable for low-complexity tasks compared to the cloud. RIoTBench (70), an IoT benchmark for stream processing systems, can measure latency, throughput, jitter, and CPU utilization for VMs that process data flow tasks. D-Cube (71) is a benchmarking tool that can profile end-to-end delay, reliability, and power consumption for any CPU designed for deploying IoT protocols on the edge.

An Intel Xeon E3-1275 v5 processor included in the Intel fog reference design (FRD) was benchmarked using CAVBench (72), which is a benchmarking program for connected and autonomous vehicles. CAVBench deploys computer vision and deep learning applications on a processor and measures the average frames per second and latency. This study concluded that deep learning applications cannot achieve satisfactory performance on the Xeon CPU, meaning accelerators are required. EdgeBench (66) utilizes audio, image, and scalar pipeline applications to evaluate the computation time, end-

[11] http://lmbench.sourceforge.net/
[12] https://nbench.io/
[13] https://github.com/akopytov/sysbench
[14] https://mycroft.ai/

to-end latency, and CPU utilization of RPi3. HERMIT (74) is a benchmarking suite for medical IoT and was utilized to test RPi3 to determine if it is suitable for medical IoT. Various Intel and ARM CPUs were benchmarked using machine learning models in pCAMP (76). pCAMP compares TensorFlow, Caffe2, MXNet, PyTorch, and TensorFlow Lite in terms of inference time, total time, and energy consumption. DeFog (10) was used to evaluate ARM-based SBCs, including RPi3 and OXU4, in various fog computing scenarios consisting of object classification and speech-to-text conversion. IoTBench (79) offers diverse IoT applications in the vision, speech recognition, and physiological signal processing domains and was used to evaluate RPi3's performance in terms of executed instructions, cycles, and cache misses.

A.2. Accelerators. Although the use of hardware accelerators has been proposed for edge computing (85), there is limited use of such accelerators in the explicit edge performance benchmarks listed in Table 2 (3 of 21 (76, 81, 82)).

pCAMP (76) benchmarks the inference time of deep learning workloads for edge computing on the Nvidia Jetson TX2. Low-power and purpose-built accelerators such as the Intel Movidius Myriad XVPU (Vision Processing Unit) (86), Google Edge TPU (87), NVIDIA 128-core Maxwell and 256-core Pascal architecture-based GPU, and custom field-programmable gate arrays (FPGAs) are benchmarked using deep neural networks on edge devices (81). During the benchmarking process, it was noted that custom FPGAs optimize the performance of specific neural network applications (82) (also refer to (88)). Various FPGA platforms have been evaluated using custom benchmarks that implement separable convolutional neural network (SCNN) keyword spotting (89). Their performances were compared to that of the Intel NCS platform in terms of inference time, power consumption, and energy per inference (82).

In addition to the abovementioned explicit edge performance benchmarks that evaluate accelerators, implicit edge performance benchmarks consider accelerators as well. The most relevant of these benchmarks are presented below.

The Intel NCS 2 and Google's Coral USB accelerators were benchmarked using popular inference workloads, namely MobileNet-v1 (90) and Inception-v1 (91), in the MLPerf benchmark (92) in terms of inference time and energy efficiency (93).

To explore whether a particular deep learning model can provide sufficient accuracy on edge devices, TomoGAN (94, 95), which is an algorithm for enhancing the quality of X-ray images, was adapted to run on the Google Edge TPU and NVIDIA Jetson TX2 (96). The benchmarking results indicated that edge accelerators can provide sufficient accuracy with a novel shallow CNN called the fine-tune network.

In a recent study, the Google Edge TPU, NVIDIA 128-core Maxwell GPU, and Intel Movidius Myriad X VPU were benchmarked by executing eight deep learning models used in personal-scale sensory systems (97). The benchmarking results revealed that the Google Edge TPU outperformed the other accelerators on all eight models. In terms of energy efficiency, the Google Edge TPU utilized less than 10 mJ of energy for a single execution of any of the eight models, whereas the other platforms consumed between 5 mJ and 274 mJ depending on the model.

A.3. Memory. Seven of the explicit benchmarks in Table 2 consider memory when benchmarking. The CoAP benchmark (67) measures memory usage and latency to evaluate the memory performance of SBCs. CAVBench (72) measures the memory bandwidths and footprints of computer vision and deep learning applications because these applications require significant memory bandwidth and can act as a bottleneck in capacity-limited edge devices. EdgeBench (66) evaluates the memory utilization of benchmark applications on the Raspberry Pi 3B. HERMIT (74) was used to analyze the memory characteristics of benchmark applications, and it was determined that a large L1 D-cache and last-level cache (LLC) are required in Raspberry Pi to achieve efficient memory access.

A few examples of implicit memory benchmarking in edge computing systems are presented below. The STREAM benchmark[15], which measures sustainable memory bandwidth, was used to evaluate edge resources (68). The memory performance of copy, scale, add, and triad operations in STREAM was measured on three different software systems: native Linux, Docker, and KVM. The Unix command mbw was utilized to test the memory performance of a wide range of ARM-based SBCs (69). The mbw command quantifies available memory bandwidth by transferring large arrays of data in memory. OC1+ outperformed RPi2 because OC1+ adopts 792 MHz LPDDR3 RAM, whereas RPi2 uses 400 MHz LPDDR2 RAM. RPi3 utilizes 900 MHz LPDDR2 RAM and outperforms OC1+ in most cases. Odroid C2 (OC2) and Odroid XU4 (OXU4) outperform RPi2, RPi3, and OC1+ based on their doubled RAM capacity.

A.4. Storage. Only two explicit edge performance benchmarks consider storage devices. Small form factor edge resources typically use flash-based storage devices, such as embedded MultiMediaCard (eMMC) and MicroSD. The storage performance of edge resources has been evaluated using Bonnie++[16] and the Unix DD command (68). Bonnie++ measures data read and write bandwidth, as well as the number of file operations per second. DD is used to measure bandwidth for accessing special device files, such as /dev/zero/. The fio benchmark[17] has been used to perform sequential read/write operations in MicroSD cards, and sysbench has been utilized to perform random disk operations on eMMC cards (69). The evaluation results revealed that eMMC cards operate at speeds in the order of hundreds of MB/s, whereas MicroSD cards operate in the range of hundreds of Mb/s.

A.5. Network. Although the network plays a key role in performance on the edge, there are only two explicit benchmarks that address this aspect. The network performance of Cubieboard2, which provides a 100BASE-TX Fast Ethernet connection, was evaluated using Netperf[18] in native, Docker, and KVM environments (68). The results revealed that Docker achieves near-native performance, whereas KVM introduces considerable overhead. DeFog (10) was used to measure communication latency from an edge device to the Amazon cloud when the network bandwidth was fully utilized by the stress-ng operation[19].

[15] https://www.cs.virginia.edu/stream/
[16] https://www.coker.com.au/bonnie++/
[17] https://github.com/axboe/fio
[18] https://hewlettpackard.github.io/netperf/
[19] https://kernel.ubuntu.com/git/cking/stress-ng.git/

Observation #1. Even though there have been considerable efforts devoted to benchmarking CPUs and moderate efforts for benchmarking memory, additional effort is still required to measure the edge performance of accelerators, storage, and networks effectively.

B. Software Platforms. Orchestrators, cloud services, schedulers, and AI platforms are examples of software platforms that are benchmarked on edge computing systems. Although other software platforms exist, there has been no research in the context of edge performance benchmarking, so these platforms are not discussed in this article.

B.1. Orchestrators. Orchestrators for edge computing manage edge resources by creating containers, deploying and starting servers, and assigning and scaling computational resources. Orchestration in edge computing is challenging because of limited hardware resources, large volumes of edge resources, and the mobility of connected devices (11).

Edgedroid (77) is the only explicit edge performance benchmark that considers orchestrators. Edgedroid evaluates human-in-the-loop applications such as augmented reality and wearable cognitive assistance that are deployed on edge devices. Such applications connect to containers in the cloud for background processing. Edgedroid collects uplink, downlink, and processing time data to identify scaling limits.

However, there are also a few implicit edge performance benchmarks that consider orchestrators. FocusStack coordinates edge resources for moving targets, such as cars and drones (98). This is a challenging task because existing orchestrators (e.g., OpenStack) were developed to manage a relatively small number of servers. FocusStack extends OpenStack to support location-based awareness to minimize the number of devices managed at one time. The full-time active monitoring system of FocusStack was benchmarked using the following metric, which represents the total number of bytes transferred every 10 s for monitoring between the orchestrator and a device. FocusStack sent 358 bytes every 10 s while the unmodified OpenStack transferred 17,509 bytes every 10 s.

Edge workload orchestrators that select target edge resources for offloading tasks have been proposed based on fuzzy logic (99). Such orchestrators can be benchmarked by relying on data obtained from the EdgeCloudSim simulator (100). The explored applications include augmented reality, healthcare, intensive computing, and infotainment applications, which are evaluated in terms of service time, failed tasks, and virtual machine utilization.

B.2. Service Model. Different service models can facilitate the delivery of services on the edge. Service models can be serverless models or Infrastructure-as-a-Service (IaaS) models. There is limited consideration for such models in the explicit edge benchmarks listed in Table 2. EdgeBench (66) provides benchmarking applications for the serverless edge computing service model and compares the performances of the AWS IoT Greengrass[20] and Azure IoT Edge[21] platforms.

Some implicit edge performance benchmarks consider the IaaS model and two examples are discussed below. Nebula implements a decentralized edge cloud by utilizing volunteer edge nodes that provide computational and storage resources (101). Nebula offers the IaaS service model (computation and storage services are available). MapReduce, Wordcount, and InvertedIndex applications have been used to benchmark Nebula in terms of performance, fault tolerance, and scalability (102).

Similarly, FemtoClouds offers IaaS-type services by forming small clusters using smartphones and laptops by leveraging idle or less-loaded resources to fulfill user requests (103). FemtoClouds has been benchmarked for various metrics, including computational throughput, network utilization, and computational resource utilization.

B.3. Schedulers. Edge schedulers allow service providers to allocate computing resources efficiently. None of the explicit edge performance benchmarks can compare the performances of edge schedulers. However, several implicit edge performance benchmarks have considered resource scheduling policies to satisfy the real-time requirements of smart manufacturing applications in edge computing (104). A two-phase scheduling strategy is adopted that first selects a suitable edge computing server based on the target task load, after which additional servers are selected, if necessary, to distribute tasks when one server cannot meet real-time constraints. Schedulers are evaluated on OpenCV applications using metrics such as computing latency, satisfaction degree, and energy consumption. The fairness aspect of edge scheduling was recently evaluated using synthetic benchmarks (105).

B.4. Artificial Intelligence (AI) Platforms. Three of the explicit edge performance benchmarks consider AI platforms. Edge AIBench (73) provides four AI benchmark applications that can reflect complex scenarios of edge computing, including intensive care unit patient monitoring, surveillance cameras, smart homes, and autonomous vehicles. These test models can be executed using a federated learning framework in a publicly available test bed[22]. LEAF (75) is a modular benchmarking framework for evaluating learning in federated settings. LEAF consists of open-source datasets, statistical and system metrics, and reference implementations. AIoTBench (65) was designed to evaluate the AI capabilities of edge devices for image classification, speech recognition, transformer translation, and micro workloads.

Observation #2. Explicit edge performance benchmarks do not consider software platforms such as orchestrators, service models, and schedulers, which are vital for performance on the edge. Therefore, existing benchmarks cannot capture the integrated performances of services or applications when different software platforms are adopted.

5. Techniques Analyzed

This section reviews various techniques that have been analyzed in edge performance benchmarks across the dimensions of application optimization and application deployment options. Table 3 categorizes the benchmarks listed in Table 1. Table 4 summarizes the considered dimensions for a selected set of implicit edge performance benchmarks.

Tables 3 and 4 list the different optimization options for target applications that are considered in explicit and implicit edge performance benchmarks, respectively. These options are

[20] https://aws.amazon.com/greengrass/

[21] https://azure.microsoft.com/en-us/services/iot-edge/

[22] http://www.benchcouncil.org/testbed.html/

Techniques analyzed by edge performance benchmarks	Application optimization options			Application deployment options	
	Energy consumption	Quality-of-Service	Hardware utilization	Offloading	Virtualization
CoAP benchmark (67), HERMIT (74)	N	Y	Y	N	N
Virt. benchmark (68)	N	N	Y	N	V, C
Virt. benchmark (69)	Y	Y	Y	N	C
Voice benchmark (70)	N	Y	N	D $\rightarrow$ E	N
RIoTBench (64)	N	Y	Y	N	V
TPCx-IoT	N	Y	N	N	N
D-Cube (71)	Y	Y	N	N	N
CAVBench (72)	N	Y	Y	N	N
EdgeBench (66)	N	Y	Y	E $\rightarrow$ C	V, C
Edge AIBench (73)	N	N	N	N	N
LEAF (75), AIoTBench (65)	N	N	N	N	N
pCAMP (76)	Y	Y	N	N	N
DeFog (10)	N	Y	N	C $\rightarrow$ E	V, C
Edgedroid (77)	N	Y	N	N	C
MAVBench (78)	Y	Y	N	D $\rightarrow$ C	N
IoTBench (79)	Y	N	N	N	N
SoftwarePilot	N	N	N	N	V, C
Machine learning accelerator benchmark (81)	Y	N	Y	N	V
Edge FPGA-based Neural Net benchmark (82)	Y	Y	N	N	N

Table 3. Comparison of the characteristics of the techniques analyzed by explicit edge performance benchmarks; Offloading: End-user Device, Edge, Cloud, or None; Virtualization: Virtual machine, Container, or None;

Techniques analyzed by edge performance benchmarks	Application optimization options			Application deployment options	
	Energy consumption	Quality-of-Service	Hardware utilization	Offloading	Virtualization
Resource allocation benchmark (106), MECO benchmark (107), RTLBB (108), EEDOA (109)	Y	N	N	$D \rightarrow E$	N
MIMO benchmark (110)	Y	N	N	$D \rightarrow C$	N
IoT benchmark (111), LBVS (112), MeFoRE (113), VEC benchmark (114)	N	Y	N	$D \rightarrow E$	N
IoT benchmark (115)	N	Y	N	$D \rightarrow E$	N
DYVERSE (9), ENORM (8)	N	Y	N	$C \rightarrow E$	V, C
ECC benchmark (116)	N	Y	N	$D \rightarrow E$	N
YEAST (117), EDAL (118)	Y	Y	N	N	N
Content delivery benchmark (119)	N	Y	Y	N	N
Cloudlet benchmark (120)	N	N	N	$C \rightarrow E$	V
Migration benchmark (121)	N	Y	N	$E \rightarrow E$	C
Cloudlet benchmark (122)	Y	Y	N	$D \rightarrow E$	N
Virt. benchmark (69)	Y	N	Y	N	C

Table 4. Comparison of the characteristics of the techniques analyzed by explicit edge performance benchmarks; Offloading: End-user Device, Edge, Cloud, or None; Virtualization: Virtual machine, Container, None

related to energy consumption, quality of service (QoS), and hardware utilization.

A.1. Energy Consumption. Energy is an important metric used in many edge performance benchmarks. The virtualization benchmark (69) compares the power consumption rates of five different SoCs acting as end-user devices. This type of evaluation is useful for estimating the battery durations of end-user devices. To select the best low-power wireless protocol, D-Cube (71) measures the power consumption of a target edge system while applying different protocols for an IoT application.

Moreover, energy consumption has been employed to evaluate machine learning packages on systems on the device-edge-cloud continuum (76). Such benchmarks are useful for optimizing packages for various edge systems. MAVBench (78) compares the energy consumption of a full-on-edge drone to that of a full-on-cloud drone. This performance benchmark provides a breakdown of the energy consumed by different components of MAVs. Similarly, IoTBench (79) evaluates the energy dedicated to different components of benchmarks. Recently, several machine learning benchmarks (81, 82) focusing on the energy consumption of accelerators have been developed.

Several implicit edge performance benchmarking studies have compared different energy consumption techniques, a few of which are discussed below.

Early efforts led to the formulation of a convex optimization problem for minimizing mobile energy consumption (107). A multi-user MEC system was considered with a mobile base station and multiple mobile devices, from which tasks were split using a threshold-based policy.

A fine-grained method for multi-resource joint optimization for the energy consumed for task offloading, sub-channel allocation, and CPU-cycle frequency was also developed (108). Energy efficiency can also be benchmarked when workloads have varying execution times on MEC servers (106). ThriftyEdge (123) is used to benchmark the performance of delay-aware task graph partitioning and virtual machine selection to minimize IoT device edge resource occupancy. Stochastic optimization for minimizing the energy consumption of task offloading while guaranteeing the average queue length of IoT applications was benchmarked in (109).

A.2. Quality-of-Service. QoS is a frequently examined metric in edge performance benchmarks that is represented by execution time, computation and communication latencies, etc. The CoAP benchmark (67) and HERMIT (74) measure the execution times of IoT applications on end-user devices. When comparing application performance across different layers in device-edge-cloud systems, the voice benchmark (70) and Edgedroid (77) break down execution times according to application components for analysis. A fine-grained study of application latency under varying edge resource availability characteristics and workloads can be performed using DeFog (10). Existing edge performance benchmarks largely focus on hardware with relatively little emphasis on benchmarking different algorithms to optimize edge systems. Therefore, example edge performance benchmarking studies that have focused on optimizing QoS based on algorithm designs are discussed below.

Because improving application QoS is a key component of edge computing (4), the evaluation of QoS is very common in edge literature. We discuss a few examples below. ENORM (8) benchmarks the benefits of offloading application services from the cloud to the edge based on the QoS of multiple applications on the same edge node. Priority-based resource scaling approaches can be benchmarked by DYVERSE, which estimates the amount of resources to be added to or removed from a running edge service (9). The metric used in this evaluation is the QoS violation rate, which is also employed for evaluating the performance of an optimization model for placing IoT services on edge resources to prevent QoS violations (115).

From the perspective of edge service users, MeFoRE (113) uses previous records of quality of experiences (QoE, such as service give-up ratio) to estimate the resources required by different users. QoE is frequently used in utility functions to optimize the performance of running application services (utility-based optimization). Moreover, the use of QoE to measure user utility for running jobs on the edge versus local execution on mobile devices has been benchmarked (124).

A.3. Hardware Utilization. The utilization of hardware on the edge is another dimension that can be considered to optimize edge performance. This aspect is also captured by edge performance benchmarks. The virtualization benchmark (68) compares the utilization of different hardware resources on an edge system to select optimal virtualization techniques. Similarly, CPU and memory utilization has been analyzed for benchmarking stream processing platforms (64) and machine learning accelerators (81).

Observation #3. It is noteworthy that the QoS captured by edge performance benchmarks is a dominant criterion for optimizing performance. However, other criteria, such as energy consumption, are not typically captured by edge performance benchmarks.

B. Application Deployment Options. Edge benchmarks have captured the performance of application deployment options. These include the direction of deployment and determining how to deploy applications on edge systems, which are considered by reviewing edge performance benchmarks for techniques analyzed for computation offloading and virtualization technologies, respectively. The dimension of deployment in the context of the execution environment is considered in Section B.

B.1. Computation Offloading. Only 4 of the 21 explicit edge performance benchmarks listed in Table 3 consider this dimension. This section reviews the computation offloading techniques that are analyzed in explicit edge performance benchmarking.

The following four directions of offloading are relevant to edge environments:

i. Cloud-to-Edge: Voice benchmark (70) characterizes the performance impact of pushing the execution of voice interaction pipelines closer to end-user devices. EdgeBench (66) offloads serverless functions from the cloud to the edge. Edge AIBench (73) and DeFog (10) break down the components of applications and offload some components from the cloud to the edge. Implicit edge performance benchmarks capture the benefits of cloud-to-edge offloading for database replication (119) and application cloning (120).

ii. Edge-to-Cloud: This is the typical direction for the internet of connected vehicles, and utility-based multi-level offloading schemes are evaluated to maximize the utility of vehicles, edge servers, and cloud servers (114). A collaborative offloading approach has been evaluated to optimize resource allocation and offloading decisions jointly (125).

iii. Edge-to-Edge: When an edge node does not have sufficient resources, it can offload (or migrate) its workload to a peer. Two approaches are typically benchmarked: (i) offload forwarding, which forwards all unprocessed workloads to neighboring edge nodes to meet service objectives (126); and (ii) service migration, which dynamically migrates services across multiple heterogeneous edge nodes (116). Service handoff approaches have been benchmarked to investigate their feasibility for supporting seamless migration to the nearest edge server when a mobile client is moving (121).

iv. Device-to-Edge: Edgedroid (77) offloads the backend of task guidance for wearable cognitive assistance from end-user devices to the edge. In addition to existing edge benchmarks, typical offloading from end-user devices to the edge has been evaluated for applications that require edge data aggregation for energy saving. For aggregating tasks, data from multiple devices are collected by an edge node for preprocessing and filtering tasks (117, 118).

B.2. Virtualization Techniques. More than 50% of the explicit edge performance benchmarks listed in Table 3 execute applications directly on hardware. Some benchmarks have investigated virtual machines (VMs) and/or containers, and implicit edge performance benchmarking has also considered unikernels.

VMs are important for edge computing in the context of cloudlets (127). Implicit benchmarking has focused on the evaluation of VMs on the edge. For example, the selection of the most suitable VM was evaluated for different types of applications (122), and a general approach for SLA-driven scheduling for placing VMs in a multi-network operator-sharing edge environment was benchmarked (112). The virtualization benchmark (68) compares the performances of VMs to those of containers on different edge systems.

Containers have been extensively evaluated for edge systems based on their reduced boot times and lower resource footprints compared to VMs (128). Among the explicit edge performance benchmarks, the container is the most frequently adopted virtualization technology for application deployment (e.g., EdgeBench (66), DeFog (10), and Edgedroid (77)). Other benchmarking efforts have highlighted the feasibility of using Docker containers[23] and Linux containers[24] as viable options for providing rapid edge deployment (8, 129).

Unikernels[25] are used for single-purpose applications that use library operating systems and are sealed to modification following deployment (130). The corresponding small resource footprints are attractive for edge computing. Unikernels and containers have been benchmarked according to the dimensions of scalability, security, and manageability for IoT applications on the edge (131).

Observation #4. It is noteworthy that the majority of existing edge performance benchmarks do not capture performance by deploying applications using virtualization techniques. Therefore, such benchmarks can capture the application performance of edge systems that only run a single application, but not those that operate in a multi-tenant edge environment.

[23] https://www.docker.com/
[24] https://linuxcontainers.org/
[25] http://unikernel.org/

6. Benchmark Runtime

This section will discuss the execution environments, deployment destinations, and test beds considered by edge performance benchmarks. The execution environments highlight the software- and data-related characteristics considered by different edge performance benchmarks at run time. Additionally, single and multiple destinations used for deployment by various benchmarks are considered. Finally, different infrastructure deployment options considered by edge performance benchmarks, namely real-world, lab-based, emulated, and simulated test bed infrastructures, are presented.

A. Execution Environment. Execution environments consist of the software packages and datasets required to execute a benchmark (132). Based on growing support from different programming languages (e.g., Python) and virtualization tools (e.g., Docker), execution environments can be reused across projects, research groups, and scientific disciplines. Open, representative, and comprehensive execution environments broaden research avenues by facilitating scientific exploration in new domains. However, execution environments differ significantly across edge computing applications because such applications change rapidly and frequently. Typically, benchmarks are designed with a narrow focus to target specific workloads by sacrificing the features required to reuse their execution environments.

Table 5 lists seven characteristics of open, representative, and comprehensive execution environments. First, software packages and datasets must be accessible to researchers outside the initial study. Clearly, benchmarks are accessible if they use only open-source software (Characteristic #1). However, benchmarks that represent complex and emergent workloads require custom and/or proprietary software. When it is necessary, such software should be clearly identified and made available (Characteristic #2). Likewise, the datasets used to drive benchmark execution should be open and easily portable across research contexts (Characteristic #5).

Benchmarks represent real workloads by mimicking specific aspects of their functionality. However, software components with limited functionality place less stress on edge devices compared to commercial-grade software. Therefore, execution environments that are at least partially composed of commercial-grade software can ensure representative demand on edge resources (Characteristic #3).

Researchers reuse execution environments by adjusting contextual settings to match their target domains. For example, researchers that study edge resources may ignore user actions related to QoS, whereas researchers that study edge-to-cloud offloading may consider multiple contextual settings for QoS. By design, comprehensive benchmarks support a wide range of settings (Characteristic #7). Execution environments that rely on traces from prior workload executions are often inflexible in terms of runtime and contextual adjustments. Traces have closed data models that cannot be easily extrapolated to what-if conditions. By contrast, execution environments that use data from complex models of simulations or model-

	Execution environment characteristics of edge benchmarks	CAVBench (72)	EdgeDroid (77)	EdgeBench (66)	IoTBench (79)	CoAP benchmark (67)	MAVBench (78)	SoftwarePilot (80)	DeFog (10)	Edge AIBench (73)	LEAF (75)	AIoT Bench (65)	HERMIT (74)	pCAMP (76)	RIoTBench (64)
Software (SW)	Reproduced using only open source SW	Y	N	N	N	Y	N	N	N	N	Y	N	Y	Y	N
	Custom/proprietary SW components clearly distinguished and accessible	N	N	Y	N	N	Y	Y	Y	Y	Y	Y	N	N	Y
	Employs commercial-grade software	Y	Y	Y	N	N	Y	Y	Y	Y	N	Y	N	Y	Y
	Consider the effects of compiler, SW runtime and contextual settings on execution	N	Y	N	N	N	Y	Y	Y	N	Y	Y	N	Y	N
Data	Data sets open, accessible and portable	Y	Y	N	Y	N	Y	Y	Y	Y	Y	Y	Y	Y	Y
	Data generation method: **T**races, **M**arkov Processes or **S**imulation	T	M	T	T	S	S	M	T	T	M	T	T	T	T
	Configure data generation to reproduce a wide range of workload conditions	N	Y	N	N	N	Y	Y	N	N	Y	N	N	N	N

Table 5. Comparison of the software and data related execution environment characteristics of edge performance benchmarks

driven stochastic methods can be ported to new domains and workload conditions (Characteristics #4 and #6).

Table 5 reveals several interesting trends. Edge benchmarks often employ commercial-grade software components (71%) and use open datasets (84%). These results are likely to be influenced by the machine learning kernels in edge workloads. Commercial-grade versions of neural network platforms, object detection models, and speech processors are open sources that are widely used. By contrast, only 50% of the studied benchmarks support comprehensive evaluation across runtime and contextual settings. This result stems from common dependence on traces from prior executions. Furthermore, 75% of the benchmarks are narrowly tailored to specific contexts and use closed-model traces that are not easily adapted across runtime and contextual settings.

Among the studied benchmarks, EdgeDroid (77, 133), SoftwarePilot (80), and MAVBench (78) have the characteristics of open, representative, and comprehensive execution environments. For example, SoftwarePilot and MAVBench capture virtual reality. Each of these benchmarks employs commercial-grade open-source software components for machine learning and cognitive assistance. Additionally, in the corresponding papers, each benchmark is evaluated across various runtime and contextual settings by adjusting the complexity of machine learning models or adapting user QoS expectations. Leaf (82) is also a comprehensive benchmark that can vary workloads and runtime settings. A subtle difference between these benchmarks is that MavBench relies on simulated environments, which is a design choice that could yield non-representative workloads if simulations deviate from real-world edge conditions. By contrast, SoftwarePilot, EdgeDroid, and Leaf use Markov decision models to adapt real data to new contextual settings. This approach is likely to be more robust for researchers targeting new domains.

Observation #5. Note that most benchmarks do not operate on data from a real-world edge test bed. Instead, they use simulation data, trace data, or data from a test bed adapted to a different contextual setting. While this issue can be attributed to a lack of readily available edge test beds, it also highlights the need for revisiting and validating existing edge performance benchmarking approaches when new edge test beds become available.

Observation #6. There is a limited selection of edge performance benchmarks that can generate data to capture a wide range of workload conditions. This factor translates into benchmarks that are narrowly focused on specific applications and cannot be used generically.

B. Deployments. This section will explore the different resource locations at which benchmarks are executed, which are referred to as deployment destinations. It will also review test bed options. The following deployment destinations are considered: (i) single destination and (ii) multiple destinations.

B.1. Destination. As mentioned previously, this article considers the edge as resources located at the edge of a wired network. However, a number of researchers have also considered user devices as the edge by adhering to a broader definition of the edge (of a network). Therefore, we consider device-only deployment for a single destination.

i. Single destination refers to benchmark execution on only one device or edge.

a. Device-only: Research that explores device-only benchmarking has been considered for benchmarking in different areas, such as low-power wireless industrial sensors, device-specific protocols, low-overhead virtualization, medical IoT devices, and devices that will execute machine-learning-based workloads.

Low-power wireless industrial sensors: Wireless protocols for industrial sensor devices have been benchmarked using observation modules (71). The six protocols considered are (i) Enhanced ContikiMAC, (ii) Thompson-sampling-based channel selection, (iii) Glossy, (iv) Chaos, (v) Sparkle, and (vi) Time-slotted channel hopping. Power consumption and end-to-end latencies are profiled, and an additional validation mechanism is incorporated to evaluate the accuracy of benchmarking.

Device-specific protocols: Some researchers have benchmarked system architectures for constrained application protocol (CoAP)-based IoT devices (67). The key results from

benchmarking can be summarized as follows: (i) the selected processor directly impacts CoAP server performance and (ii) the latency of communication channels affects the round-trip times of CoAP requests.

Low-overhead virtualization: The performance of virtualization for user devices has been benchmarked extensively (68, 69). A range of virtualization techniques such as docker containers and kernel-based virtual machines (KVMs) have been considered (68). The key results are that hypervisor-based virtualization incurs large overhead and containers seem to be more appropriate for network edges. Container-based virtualization across five different devices has also been considered (69). Moreover, there is a negligible impact on performance when using containers compared to bare-metal execution. The characteristics of workloads represent key information for estimating the energy efficiency of devices.

Medical IoT devices: The computation and memory characteristics of IoT devices used in the medical domain have been benchmarked (74). A collection of medical applications (macro benchmarks) was evaluated against the MiBench, PARSEC, and CPU06 micro benchmarks. The evaluation results revealed that execution characteristics differ between micro and macro benchmarks.

Devices that run machine learning workloads: The benchmarking of machine-learning-specific workloads for various devices was presented in (65). Both micro benchmarks, such as the individual layers of a neural network, and macro benchmarks, such as applications in image classification, speech recognition, and language translation on the TensorFlow and Caffe2 frameworks, were considered. Similarly, benchmarking for the vision and speech domains has also been considered (79).

b. Edge-only: Research that explores benchmarking for edge-only deployment will ideally utilize macro benchmarks that are edge native (edge resources are not merely accelerators, but are essential for an application to be used in the real world). Autonomous vehicles are one such application. CAVBench is a benchmarking suite developed for autonomous vehicles by focusing on real-time applications that must process unstructured data (72). The edge node employed in this research was designed based on the Intel fog reference design. Hardware accelerators, such as FPGAs on the edge, have been benchmarked for convolutional neural networks in the context of keyword spotting (82). It is worth nothing that this use case may not necessarily represent an edge-native benchmark.

ii. Multiple-destination: This refers to the execution of a benchmark either across an entire device-edge-cloud resource pipeline or a partial resource pipeline (e.g., device-edge or edge-cloud).

a. Entire device-edge-cloud pipeline: There are a number of examples of benchmarking studies that leverage an entire resource pipeline consisting of a device, the edge of a wired network, and a cloud for benchmarking. Three main types of applications are considered.

Service pushdown: Voice-interactive applications have been used as macro benchmarks for analyzing entire resource pipelines (70). The goal is to push services from the cloud across weak devices and the edge to optimize applications to obtain consistent dialog latency.

Data aggregation: Benchmarks that are relevant to the data-intensive workflows of power grids have been presented in the context of an entire resource pipeline (TPCx-IoT)[26]. This workflow enables real-time analytics to be performed on gateways while ingesting data from 200 different types of power station sensors. This benchmark operates in two runs: one run for a warm up and another for measurement. Performance, price, and availability metrics are considered. The majority of benchmarks considered by DeFog fall under this category (10).

Edge inference: Benchmarks are employed to train machine learning models on the cloud and perform inference on the edge (assuming that a trained model is available on the edge) (73). A variety of devices or sensors are considered to generate data in patient monitoring, surveillance, or smart home scenarios. Similarly, inference on a device, edge, or cloud for different machine learning platforms, such as TensorFlow, Caffe2, PyTorch, MXNet, and TensorFlowLite, is considered by pCAMP (76), which can also consider different accelerators (81). It should be noted that in this case, inference is not distributed because it is performed entirely on the edge.

b. Partial pipeline: The following three combinations for benchmarking on partial resource pipelines are considered.

Device-Edge: Edgedroid (77) is an example of benchmarking human-in-the-loop applications, such as cognitive assistance in an edge-cloud deployment, where the edge is a cloudlet. The underlying benchmarking approach is to mimic applications by replaying traces of sensory inputs that are obtained from running the application in the real world. The feedback generated by processing such sensory inputs on the cloudlet is processed using a model of human reactions. This enhances the understanding of latency tradeoffs in the contexts of both the application and edge-cloud deployment.

Device-Cloud: The device-cloud pipeline focuses on estimating the performance of distributed intelligence systems (134) and IoT stream application compositions (64). The former type of system deploys a sliced neural network across a user device and cloud for distributed inference by estimating where a neural network should be sliced. The latter facilitates benchmarking by combining distributed stream applications using modular IoT tasks.

Edge-Cloud: This deployment pipeline is typically used to investigate the performance of applications and the lifecycle of application service offloading from the cloud to the edge and vice-versa (although this is not exclusive to the edge-cloud pipeline and is also relevant in other partial pipelines and the entire resource pipeline (refer to Section B.1)). Benchmarks that exploit this pipeline include EdgeBench (66) and SoftwarePilot (80).

Observation #7. A number of explicit edge performance benchmarks focus on either a device or edge as a deployment destination. There are relatively few edge performance benchmarks that consider the entire resource pipeline, which integrates a cloud, edge, and device.

B.2. Test beds. The test bed options for deploying edge benchmarks include real-world and physical, lab-based experimental, emulated, or simulated infrastructures.

i. Real-world physical infrastructure: This refers to "in the wild" physical test beds that closely mimic the operational characteristics of an actual edge deployment. There are only a

[26] http://www.tpc.org/tpc_documents_current_versions/pdf/tpcx-iot_v1.0.4.pdf

few such test beds available, such as the Living Edge Lab[27] and those reported in (135, 136). There has been no evaluation of any of the benchmarks listed in Table 1 on real-world infrastructures.

ii. Lab-based experimental infrastructure: The majority of test beds on which the edge applications or benchmarks listed in Table 1 have been evaluated are lab-based infrastructures. Such test beds may be public cloud offerings or private cloud resources coupled with edge resources in the form of single-board computers (8, 10) (or a cluster (137)) or routers/gateways. End user devices may range from wireless sensors (71) to user gadgets (8, 70). Moreover, a number of benchmarks do not rely on real data, but instead use simulation data based on the complexity of the environments they benchmark (e.g., autonomous cars (72) or drones (80)).

iii. Emulated infrastructure: Real-world physical infrastructure is not easily accessible to many researchers, and many lab-based experimental infrastructures are small in scale and do not represent the characteristics of a real infrastructure. Therefore, various types of emulated infrastructures have recently been proposed (138). An emulated environment relies on deploying edge servers on the cloud and configuring these servers, as well as the network and interconnects, such that they represent real edge environments. However, this method assumes knowledge of the parameters required to configure a realistic emulated edge environment, which can only be acquired from a real-world edge infrastructure.

iv. Simulation infrastructure: A number of simulators, such as EdgeCloudSim (100), iFogSim (139), FogExplorer (140, 141), and MyiFogSim (142), are available for edge computing and can provide insights into basic design choices. However, more complex integration tests and application component-specific analysis cannot be performed using such simulators. Therefore, such methods should only be used as a fallback solution when real or emulated benchmarking infrastructures are not available.

Observation #8. Following observation #5, we note that most edge performance benchmarking is conducted on experimental, emulated, or simulation-based infrastructures that may not be representative of large and geo-distributed real-world physical infrastructures.

7. Future Directions and Conclusions

This survey provided a catalog of explicit edge performance benchmarks and a subset of implicit edge performance benchmarking research to achieve objective O1 stated in Section A. We presented a brief timeline of performance benchmarking for different computing systems and then considered edge performance benchmarking to achieve objective O2 stated in Section A. The key dimensions for edge performance benchmark categorization include the system under test, techniques analyzed, and benchmark runtime, which were examined to achieve objective O3 stated in Section A. In exploring the key dimensions of edge performance benchmarks, we addressed the three key research questions posed in Section A. Furthermore, we highlighted eight observations relevant to the scope of this paper. The final objective (O4 in Section A) of presenting future research directions is accomplished in this section. The eight observations will be mapped onto future research directions, and the general areas of research will be discussed.

Eight avenues for pursuing future edge performance benchmarking are presented below.

i. Widening the scale of geo-distribution: Following Observation #8, most current edge performance benchmarking research is not conducted on real test beds with geo-distributed infrastructures. Many existing benchmarks are designed to capture the performances of individual cloud or edge resources in lab-based test beds. Therefore, these benchmarks do not necessarily capture the performance of large-scale geo-distribution that will be observed in an edge computing environment. The absence of comprehensive edge datasets has been also reported previously (143). Although existing performance benchmarks have provided important insights, more comprehensive edge performance benchmarks must fully embrace geo-distributed benchmarking for large-scale collections of cloud and edge resources.

ii. Developing edge-specific quality and performance metrics and measurement techniques on different platforms: As noted in Observation #2, current edge performance benchmarks do not capture performance on different software platforms, such as orchestrators, schedulers, and service models. To evaluate such software platforms, multiple multi-instance workloads with workflows for the relevant applications are required to test the resource provisioning and sharing capabilities of edge platforms. Addressing this issue would provide researchers with a valuable tool for quantifying the performance of edge applications that will run on a variety of platforms. Additionally, current efforts largely focus on capturing metrics that are historically relevant to distributed systems, such as grids and clouds. Although such metrics are useful, it is likely that edge-specific metrics considering the transient and massively dispersed nature of such environments have not yet been developed. Additionally, novel techniques to capture these metrics may be required.

iii. Evaluating benchmarks on real-world infrastructures: Lab-based infrastructures are the most common test beds used to evaluate edge benchmarks, as noted in Observation #5. At least in part, this can be attributed to limited access to real test beds. Regardless, additional efforts are required to evaluate existing benchmarks on real and resource-rich test beds to identify limitations in current benchmarking approaches.

iv. Developing lightweight and rapid edge benchmarks that capture application performance in multi-tenant environments: Generally, edge and mobile resources have limited capabilities to execute common and extensive benchmark applications designed for large data center servers. Many HPC applications have been used to capture the performance of CPUs and accelerators for edge computing, but such methods are very time consuming. Additionally, running unmodified Spark or Hadoop applications for big data platforms requires significant time and resources to obtain useful results. Therefore, lightweight benchmarking in terms of actual benchmarks and measurement techniques must be designed and developed for edge platforms. As noted in Observation #4, many current edge performance benchmarks execute a single application without considering virtualization. Recent edge systems are designed to utilize virtual machines and containers to support multi-tenancy (11). Thus, there is a need to design and de-

[27] https://www.openedgecomputing.org/living-edge-lab/

velop lightweight and rapid edge benchmarks for multi-tenant edge environments that can quantify the impact of multiple concurrent users competing for the same resources.

v. Developing standardized benchmarks across the entire resource pipeline for capturing offloading performance and varying workload conditions: The premise of edge computing is to offload tasks either from a cloud or devices to an edge to reduce the overall response time of an application and improve energy efficiency. Most edge performance benchmarks do not capture the performance of an entire resource pipeline (devices, edges, and clouds), as noted in Observation #7, meaning they do not capture the performance of offloading coherently. Many evaluations presented in the existing literature have used various workloads and metrics relevant to specific platforms or test beds. Therefore, they are non-standard benchmarks and are not compatible across different infrastructures. Additionally, as noted in Observation #5, existing edge performance benchmarks have limited flexibility in terms of capturing a wide range of workload conditions. Hence, a more comprehensive and standard approach for benchmarking offloading mechanisms and varying workload conditions must be considered.

vi. Maturing edge performance benchmarking: Current edge performance benchmarks typically consider QoS metrics, while other relevant criteria, such as energy consumption, are not considered, as noted in Observation #3. Additionally, most metrics focus on CPUs, but additional research is required to quantify the performance of accelerators, storage, and networks at the edge, as noted in Observation #1. A benchmarking suite that can holistically capture CPU, accelerator, memory, storage, and network performance on edge platforms and generate performance scores that are normalized against reference platforms is required. These are a few relevant areas that must receive additional attention from the research community to advance edge performance benchmarking research.

vii. Moving beyond performance in edge benchmarking: In addition to performance in benchmarking, there are other quality dimensions such as elastic scalability, data consistency, security and privacy, and availability that are typically in direct or indirect tradeoff relationships (13, 144). Current *edge* benchmarking approaches do not consider quality dimensions beyond performance. Benchmarking approaches from other closely related domains, such as cloud computing, can often be adapted or reused for edge environments. For example, there are large numbers of starting points for elastic scalability (38, 60, 145–148), availability (149–155), data consistency (45, 46, 144, 156–160), and security and privacy (54, 161–167). A more detailed discussion of additional quality dimensions for edge benchmarking is beyond the scope of this paper.

viii. Security/privacy-specific edge benchmarks: Edge systems are significantly more complex than previous iterations of distributed systems (volume of devices connected, heterogeneity of resources and technological domains spanned, and edge resources that are accessible for computing, which were previously unavailable or concealed within networks). This complexity naturally creates a large attack surface and multiple vulnerable spots in terms of data privacy. Therefore, benchmarks that provide insights into identifying security mechanisms for orchestrating services and suitable security standards for complex systems would be very useful.

Relevance of edge performance benchmarking to practitioners: Edge performance benchmarking is a nascent topic within edge computing research. We anticipate that multiple practitioner groups will benefit from edge performance benchmarks. We list four relevant groups below (10, 13). (i) Edge hardware vendors can tabulate and demonstrate the advantages of edge computing by using performance benchmarks. (ii) System software administrators can investigate the effects on edge applications when changes are introduced within an edge compute infrastructure, such as updates or patches to operating systems, system software, or runtime libraries. (iii) Service providers can select the most appropriate geographic locations for deploying micro and modular data centers on the edge and can quantify the performance of specific applications to justifying their choice of location. (iv) Network administrators may wish to quantify edge application performance when changes are introduced in the network stack, such as a new network protocol or security patch in a specific layer of the stack.

Additionally, real-time edge performance benchmarking can be integrated with automated edge software development and adaptive edge orchestration platforms to select the most appropriate edge resource for deployment based on current performance and network conditions. In this context, edge performance benchmarks will be of significant interest to any edge application developer.

ACKNOWLEDGMENTS. The first author is supported by funds from Rakuten Mobile, Japan and by a Royal Society Short Industry Fellowship.

1. B Varghese, R Buyya, Next Generation Cloud Computing: New Trends and Research Directions. *Futur. Gener. Comput. Syst.* **79**, 849–861 (2018).
2. I Foster, C Kesselman, eds., *The Grid: Blueprint for a New Computing Infrastructure.* (Morgan Kaufmann Publishers Inc.), (1998).
3. B Varghese, et al., Cloud Futurology. *Computer* **52**, 68–77 (2019).
4. B Varghese, N Wang, S Barbhuiya, P Kilpatrick, DS Nikolopoulos, Challenges and Opportunities in Edge Computing in *Proceedings of the IEEE International Conference on Smart Cloud.* pp. 20–26 (2016).
5. W Shi, J Cao, Q Zhang, Y Li, L Xu, Edge Computing: Vision and Challenges. *IEEE Internet Things J.* **3**, 637–646 (2016).
6. F Pallas, P Raschke, D Bermbach, Fog Computing as Privacy Enabler. *IEEE Internet Comput.* (2020).
7. M Satyanarayanan, The Emergence of Edge Computing. *Computer* **50**, 30–39 (2017).
8. N Wang, B Varghese, M Matthaiou, DS Nikolopoulos, ENORM: A Framework for Edge Node Resource Management. *IEEE Transactions on Serv. Comput.* (2017).
9. N Wang, M Matthaiou, DS Nikolopoulos, B Varghese, DYVERSE: DYnamic VERtical Scaling in Multi-tenant Edge Environments. *Futur. Gener. Comput. Syst.* **108**, 598–612 (2020).
10. J McChesney, N Wang, A Tanwer, E de Lara, B Varghese, DeFog: Fog Computing Benchmarks in *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing.* p. 47–58 (2019).
11. CH Hong, B Varghese, Resource Management in Fog/Edge Computing: A Survey on Architectures, Infrastructure, and Algorithms. *ACM Comput. Surv.* **52** (2019).
12. D Bermbach, et al., A research perspective on fog computing in *Proceedings of the 2nd Workshop on IoT Systems Provisioning and Management for Context-Aware Smart Cities.* (2017).
13. D Bermbach, E Wittern, S Tai, *Cloud Service Benchmarking: Measuring Quality of Cloud Services from a Client Perspective.* (Springer), (2017).
14. A Elhabbash, F Samreen, J Hadley, Y Elkhatib, Cloud Brokerage: A Systematic Survey. *ACM Comput. Surv.* **51** (2019).
15. HJ Curnow, BA Wichmann, A Synthetic Benchmark. *The Comput. J.* **19**, 43–49 (1976).
16. RP Weicker, Dhrystone: A Synthetic Systems Programming Benchmark. *Commun. ACM* **27**, 1013–1030 (1984).
17. JJ Dongarra, P Luszczek, A Petitet, The LINPACK Benchmark: Past, Present and Future. *Concurr. Comput. Pract. Exp.* **15**, 803–820 (2003).
18. DH Bailey, et al., The NAS Parallel Benchmarks—Summary and Preliminary Results in *Proceedings of the ACM/IEEE Conference on Supercomputing.* p. 158–165 (1991).
19. R Eigenmann, S Hassanzadeh, Benchmarking with Real Industrial Applications: The SPEC High-Performance Group. *IEEE Comput. Sci. Eng.* **3**, 18–23 (1996).
20. S Sharma, C.-H. Hsu, W.-C. Feng, Making a Case for a Green500 List in *Proceedings of the 20th IEEE International Parallel Distributed Processing Symposium.* (2006).
21. PR Luszczek, et al., The HPC Challenge (HPCC) Benchmark Suite in *Proceedings of the ACM/IEEE Conference on Supercomputing.* (2006).
22. JD McCalpin, Memory Bandwidth and Machine Balance in Current High Performance Computers. *IEEE Tech. Comm. on Comput. Archit. Newsl.* (1995).
23. MA Heroux, JJ Dongarra, Toward a New Metric for Ranking High Performance Computing Systems, (Sandia National Lab), Technical Report SAND2013-4744 (2013).
24. S Che, et al., Rodinia: A Benchmark Suite for Heterogeneous Computing in *Proceedings of the IEEE International Symposium on Workload Characterization.* pp. 44–54 (2009).

25. A Danalis, et al., The Scalable Heterogeneous Computing (SHOC) Benchmark Suite in *Proceedings of the 3rd Workshop on General-Purpose Computation on Graphics Processing Units*. p. 63–74 (2010).
26. B Hu, CJ Rossbach, Mirovia: A Benchmarking Suite for Modern Heterogeneous Computing. *CoRR* **abs/1906.10347** (2019).
27. A Snavely, G Chun, H Casanova, RFV der Wijngaart, M Frumkin, Benchmarks for Grid Computing: A Review of Ongoing Efforts and Future Directions. *SIGMETRICS Perform. Eval. Rev.* **30**, 27–32 (2003).
28. M Frumkin, RF Van der Wijngaart, NAS Grid Benchmarks: A Tool for Grid Space Exploration in *Proceedings of the 10th IEEE International Symposium on High Performance Distributed Computing*. pp. 315–322 (2001).
29. G Tsouloupas, MD Dikaiakos, GridBench: A Tool for Benchmarking Grids in *Proceedings of the 4th International Workshop on Grid Computing*. (2003).
30. C Dumitrescu, I Raicu, M Ripeanu, I Foster, DiPerF: An Automated Distributed Performance Testing Framework in *Proceedings of the IEEE/ACM International Workshop on Grid Computing*. pp. 289–296 (2004).
31. A Iosup, D Epema, GRENCHMARK: A Framework for Analyzing, Testing, and Comparing Grids in *Proceedings of the IEEE International Symposium on Cluster Computing and the Grid*. pp. 313–320 (2006).
32. MI Andreica, et al., Towards ServMark, an Architecture for Testing Grid Services. *Technical University of Delft - Technical Report ServMark-2006-002, July 2006* (2006).
33. F Nadeem, R Prodan, T Fahringer, A Iosup, *Benchmarking Grid Applications*. (Springer US, Boston, MA), pp. 19–37 (2008).
34. X Yang, X Li, Y Ji, M Sha, CROWNBench: A Grid Performance Testing System Using Customizable Synthetic Workload in *Progress in WWW Research and Development*, eds. Y Zhang, G Yu, E Bertino, G Xu. (Springer Berlin Heidelberg), pp. 190–201 (2008).
35. A Gaikwad, V Doan, M Bossy, F Baude, F Abergel, SuperQuant Financial Benchmark Suite for Performance Analysis of Grid Middlewares in *Modeling, Simulation and Optimization of Complex Processes*, eds. HG Bock, XP Hoang, R Rannacher, JP Schlöder. (Springer Berlin Heidelberg), pp. 103–113 (2012).
36. B Schroeder, A Wierman, M Harchol-Balter, Open Versus Closed: A Cautionary Tale in *Proceedings of the 3rd Conference on Networked Systems Design & Implementation*. (2006).
37. BF Cooper, A Silberstein, E Tam, R Ramakrishnan, R Sears, Benchmarking Cloud Serving Systems with YCSB in *Proceedings of the 1st ACM Symposium on Cloud Computing*. p. 143–154 (2010).
38. D Kossmann, T Kraska, S Loesing, An Evaluation of Alternative Architectures for Transaction Processing in the Cloud in *Proceedings of the 30th International Conference on Management of Data*. pp. 579–590 (2010).
39. S Patil, et al., YCSB++: Benchmarking and Performance Debugging Advanced Features in Scalable Table Stores in *Proceedings of the 2nd Symposium on Cloud Computing*. pp. 9:1–9:14 (2011).
40. A Iosup, et al., Performance Analysis of Cloud Computing Services for Many-Tasks Scientific Computing. *IEEE Transactions on Parallel Distributed Syst.* **22**, 931–945 (2011).
41. B Varghese, O Akgun, I Miguel, L Thai, A Barker, Cloud Benchmarking for Performance in *Proceedings of the IEEE International Conference on Cloud Computing Technology and Science*. pp. 535–540 (2014).
42. B Varghese, O Akgun, I Miguel, L Thai, A Barker, Cloud Benchmarking for Maximising Performance of Scientific Applications. *IEEE Transactions on Cloud Comput.* **7**, 170–182 (2019).
43. AH Borhani, P Leitner, BS Lee, X Li, T Hung, WPress: An Application-Driven Performance Benchmark for Cloud-Based Virtual Machines. *Proc. IEEE 18th Int. Enterp. Distributed Object Comput. Conf.*, 101–109 (2014).
44. L Gillam, B Li, J O'Loughlin, APS Tomar, Cloud Benchmarking for Maximising Performance of Scientific Applications. *J. Cloud Comput. Adv. Syst. Appl.* **2** (2013).
45. D Bermbach, S Tai, Eventual Consistency: How Soon is Eventual? An Evaluation of Amazon S3's Consistency Behavior in *Proceedings of the 6th Workshop on Middleware for Service Oriented Computing*. pp. 1:1–1:6 (2011).
46. D Bermbach, S Tai, Benchmarking eventual consistency: Lessons learned from long-term experimental studies in *Proceedings of the 2nd International Conference on Cloud Engineering*. pp. 47–56 (2014).
47. A Bond, D Johnson, G Kopczynski, HR Taheri, Profiling the Performance of Virtualized Databases with the TPCx-V Benchmark in *Performance Evaluation and Benchmarking: Traditional to Big Data to Internet of Things*, eds. R Nambiar, M Poess. (Springer International Publishing), pp. 156–172 (2016).
48. R Nambiar, et al., Introducing TPCx-HS: The First Industry Standard for Benchmarking Big Data Systems in *Performance Characterization and Benchmarking. Traditional to Big Data*, eds. R Nambiar, M Poess. (Springer International Publishing), pp. 1–12 (2015).
49. C Luo, et al., CloudRank-D: Benchmarking and Ranking Cloud Computing Systems for Data Processing Applications. *Front. Comput. Sci.* **6**, 347–362 (2012).
50. M Ferdman, et al., Clearing the Clouds: A Study of Emerging Scale-out Workloads on Modern Hardware. *ACM SIGPLAN Notices* **47**, 37–48 (2012).
51. I Drago, E Bocchi, M Mellia, H Slatman, A Pras, Benchmarking Personal Cloud Storage in *Proceedings of the Internet Measurement Conference*. p. 205–212 (2013).
52. J Kuhlenkamp, K Rudolph, D Bermbach, AISLE: Assessment of Provisioned Service Levels in Public IaaS-Based Database Systems in *Proceedings of the 13th International Conference on Service-Oriented Computing*. pp. 154–168 (2015).
53. DE Difallah, A Pavlo, C Curino, P Cudre-Mauroux, OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases. *Proc. VLDB Endow.* **7**, 277–288 (2013).
54. S Müller, D Bermbach, S Tai, F Pallas, Benchmarking the Performance Impact of Transport Layer Security in Cloud Database Systems in *Proceedings of the 2nd International Conference on Cloud Engineering*. (IEEE), (2014).
55. J Kuhlenkamp, M Klems, O Röss, Benchmarking Scalability and Elasticity of Distributed Database Systems in *Proceedings of the International Conference on Very Large Databases*. pp. 1219–1230 (2014).
56. B Varghese, LT Subba, L Thai, A Barker, Container-Based Cloud Virtual Machine Benchmarking in *Proceedings of the IEEE International Conference on Cloud Engineering*. pp. 192–201 (2016).
57. T Palit, Yongming Shen, M Ferdman, Demystifying Cloud Benchmarking in *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software*. pp. 122–132 (2016).
58. H Wu, F Liu, RB Lee, Cloud Server Benchmark Suite for Evaluating New Hardware Architectures. *IEEE Comput. Archit. Lett.* (2016).
59. D Bermbach, et al., BenchFoundry: A Benchmarking Framework for Cloud Storage Services in *Proceedings of the 15th International Conference on Service Oriented Computing*. (2017).
60. Y Gan, et al., An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems*. p. 3–18 (2019).
61. M Grambow, L Meusel, E Wittern, D Bermbach, Benchmarking Microservice Performance: A Pattern-based Approach in *Proceedings of the 35th ACM Symposium on Applied Computing*. (2020).
62. AV Hoorn, J Waller, W Hasselbring, Kieker: A Framework for Application Performance Monitoring and Dynamic Software Analysis in *Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering*. pp. 247–248 (2012).
63. M Grambow, F Lehmann, D Bermbach, Continuous Benchmarking: Using System Benchmarking in Build Pipelines in *Proceedings of the 1st Workshop on Service Quality and Quantitative Evaluation in new Emerging Technologies*. (2019).
64. A Shukla, S Chaturvedi, Y Simmhan, RIoTBench: An IoT Benchmark for Distributed Stream Processing Systems. *Concurr. Comput. Pract. Exp.* **29**, e4257 (2017).
65. C Luo, et al., AIoT Bench: Towards Comprehensive Benchmarking Mobile and Embedded Device Intelligence in *Benchmarking, Measuring, and Optimizing*, eds. C Zheng, J Zhan. (Springer International Publishing), pp. 31–35 (2019).
66. A Das, S Patterson, MP Wittie, EdgeBench: Benchmarking Edge Computing Platforms in *Proceedings of the 4th International Workshop on Serverless Computing*. (2018).
67. CP Kruger, GP Hancke, Benchmarking Internet of Things Devices in *Proceedings of the 12th IEEE International Conference on Industrial Informatics*. pp. 611–616 (2014).
68. F Ramalho, A Neto, Virtualization at the Network Edge: A Performance Comparison in *Proceedings of the IEEE 17th International Symposium on A World of Wireless, Mobile and Multimedia Networks*. pp. 1–6 (2016).
69. R Morabito, Virtualization on Internet of Things Edge Devices With Container Technologies: A Performance Evaluation. *IEEE Access* **5**, 8835–8850 (2017).
70. S Sridhar, ME Tolentino, Evaluating Voice Interaction Pipelines at the Edge in *Proceedings of the IEEE International Conference on Edge Computing*. pp. 248–251 (2017).
71. M Schuundefined, CA Boano, M Weber, K Römer, A Competition to Push the Dependability of Low-Power Wireless Protocols to the Edge in *Proceedings of the International Conference on Embedded Wireless Systems and Networks*. p. 54–65 (2017).
72. Y Wang, S Liu, X Wu, W Shi, CAVBench: A Benchmark Suite for Connected and Autonomous Vehicles in *Proceedings of the IEEE/ACM Symposium on Edge Computing*. pp. 30–42 (2018).
73. T Hao, et al., Edge AIBench: Towards Comprehensive End-to-end Edge Computing Benchmarking in *Proceedings of the First BenchCouncil International Symposium on Benchmarking, Measuring, and Optimizing*. pp. 23–30 (2018).
74. A Limaye, T Adegbija, HERMIT: A Benchmark Suite for the Internet of Medical Things. *IEEE Internet Things J.* **5**, 4212–4222 (2018).
75. S Caldas, et al., LEAF: A Benchmark for Federated Settings. *CoRR* **abs/1812.01097** (2018).
76. X Zhang, Y Wang, W Shi, pCAMP: Performance Comparison of Machine Learning Packages on the Edges in *Proceedings of the USENIX Workshop on Hot Topics in Edge Computing*. (2018).
77. MOJOM noz, J Wang, M Satyanarayanan, J Gross, EdgeDroid: An Experimental Approach to Benchmarking Human-in-the-Loop Applications in *Proceedings of the 20th International Workshop on Mobile Computing Systems and Applications*. p. 93–98 (2019).
78. B Boroujerdian, et al., MAVBench: Micro Aerial Vehicle Benchmarking in *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture*. p. 894–907 (2018).
79. C Lee, M Lin, C Yang, Y Chen, IoTBench: A Benchmark Suite for Intelligent Internet of Things Edge Devices in *Proceedings of the IEEE International Conference on Image Processing*. pp. 170–174 (2019).
80. J Boubin, N Babu, C Stewart, J Chumley, S Zhang, Managing Edge Resources for Fully Autonomous Aerial Systems in *Proceedings of the ACM Symposium on Edge Computing*. (2019).
81. A Reuther, et al., Survey and Benchmarking of Machine Learning Accelerators in *Proceedings of the 24th Annual IEEE High Performance Extreme Computing Conference*. (2019).
82. G Dinelli, G Meoni, E Rapuano, G Benelli, L Fanucci, An FPGA-Based Hardware Accelerator for CNNs Using On-Chip Memories Only: Design and Benchmarking with Intel Movidius Neural Compute Stick. *Int. J. Reconfigurable Comput.* (2019).
83. N Rajovic, A Rico, N Puzovic, C Adeniyi-Jones, A Ramirez, Tibidabo: Making the Case for an ARM-based HPC System. *Futur. Gener. Comput. Syst.* **36**, 322–334 (2014).
84. T Davies, C Karlsson, H Liu, C Ding, Z Chen, High Performance LINPACK Benchmark: A Fault Tolerant Implementation without Checkpointing in *Proceedings of the International Conference on Supercomputing*. pp. 162–171 (2011).
85. B Varghese, C Reaño, F Silla, Accelerator Virtualization in Fog Computing: Moving from the Cloud to the Edge. *IEEE Cloud Comput.* **5**, 28–37 (2018).
86. MH Ionica, D Gregg, The movidius myriad architecture's potential for scientific computing. *IEEE Micro* **35**, 6–14 (2015).
87. S Cass, Taking AI to the Edge: Google's TPU now Comes in a Maker-friendly Package. *IEEE Spectr.* **56**, 16–17 (2019).
88. S Mittal, Survey of FPGA-based Accelerators for Convolutional Neural Networks. *Neural Comput. Appl.*, 1–31 (2018).

89. G Benelli, G Meoni, L Fanucci, A Low Power Keyword Spotting Algorithm for Memory Constrained Embedded Systems in *Proceedings of the 2018 IFIP/IEEE International Conference on Very Large Scale Integration*. pp. 267–272 (2018).
90. AG Howard, et al., Mobilenets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv preprint arXiv:1704.04861* (2017).
91. C Szegedy, et al., Going Deeper with Convolutions in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. pp. 1–9 (2015).
92. VJ Reddi, et al., MLPerf Inference Benchmark. *arXiv preprint arXiv:1911.02549* (2019).
93. LA Libutti, FD Igual, L Pinuel, LD Giusti, M Naiouf, Benchmarking Performance and Power of USB Accelerators for Inference with MLPerf in *Proceedings of the 2nd Workshop on Accelerated Machine Learning*. (2020).
94. Z Liu, et al., Tomogan: Low-dose X-ray Tomography with Generative Adversarial Networks. *arXiv preprint arXiv:1902.07582* (2019).
95. Z Liu, T Bicer, R Kettimuthu, I Foster, Deep Learning Accelerated Light Source Experiments in *Proceedings of the IEEE/ACM Third Workshop on Deep Learning on Supercomputers*. pp. 20–28 (2019).
96. V Abeykoon, Z Liu, R Kettimuthu, G Fox, I Foster, Scientific Image Restoration Anywhere in *Proceedings of the IEEE/ACM 1st Annual Workshop on Large-scale Experiment-in-the-Loop Computing*. pp. 8–13 (2019).
97. M Antonini, et al., Resource Characterisation of Personal-Scale Sensing Models on Edge Accelerators in *Proceedings of the First International Workshop on Challenges in Artificial Intelligence and Machine Learning for Internet of Things*. pp. 49–55 (2019).
98. B Amento, et al., FocusStack: Orchestrating Edge Clouds Using Location-Based Focus of Attention in *Proceedings of IEEE/ACM Symposium on Edge Computing*. pp. 179–191 (2016).
99. C Sonmez, A Ozgovde, C Ersoy, Fuzzy Workload Orchestration for Edge Computing. *IEEE Transactions on Netw. Serv. Manag.* **16**, 769–782 (2019).
100. C Sonmez, A Ozgovde, C Ersoy, EdgeCloudSim: An Environment for Performance Evaluation of Edge Computing Systems in *International Conference on Fog and Mobile Edge Computing*. pp. 39–44 (2017).
101. M Ryden, K Oh, A Chandra, J Weissman, Nebula: Distributed Edge Cloud for Data Intensive Computing in *IEEE International Conference on Cloud Engineering*. pp. 57–66 (2014).
102. D Komosny, et al., On Geographic Coordinates of PlanetLab Europe in *Proceedings of the 38th International Conference on Telecommunications and Signal Processing*. pp. 642–646 (2015).
103. K Habak, M Ammar, KA Harras, E Zegura, Femto Clouds: Leveraging Mobile Devices to Provide Cloud Service at the Edge in *Proceedings of the IEEE 8th International Conference on Cloud Computing*. pp. 9–16 (2015).
104. X Li, et al., A Hybrid Computing Solution and Resource Scheduling Strategy for Edge Computing in Smart Manufacturing. *IEEE Transactions on Ind. Informatics* **15**, 4225–4234 (2019).
105. A Madej, N Wang, N Athanasopoulos, R Ranjan, B Varghese, Priority-based Fair Scheduling in Edge Computing in *Proceedings of the International Conference on Fog and Edge Computing*. (2020).
106. J Guo, Z Song, Y Cui, Z Liu, Y Ji, Energy-efficient Resource Allocation for Multi-user Mobile Edge Computing in *Proceedings of the IEEE Global Communications Conference*. pp. 1–7 (2017).
107. C You, K Huang, H Chae, BH Kim, Energy-efficient Resource Allocation for Mobile-Edge Computation Offloading. *IEEE Transactions on Wirel. Commun.* **16**, 1397–1411 (2016).
108. P Zhao, H Tian, C Qin, G Nie, Energy-saving Offloading by Jointly Allocating Radio and Computational Resources for Mobile Edge Computing. *IEEE Access* **5**, 11255–11268 (2017).
109. Y Chen, et al., Energy Efficient Dynamic Offloading in Mobile Edge Computing for Internet of Things. *IEEE Transactions on Cloud Comput.* (2019).
110. S Sardellitti, G Scutari, S Barbarossa, Joint Optimisation of Radio and Computational Resources for Multicell Mobile-Edge Computing. *IEEE Transactions on Signal Inf. Process. over Networks* **1**, 89–103 (2015).
111. O Skarlat, M Nardelli, S Schulte, M Borkowski, P Leitner, Optimized IoT Service Placement in the Fog. *Serv. Oriented Comput. Appl.* **11**, 427–443 (2017).
112. K Katsalis, T Papaioannou, N Nikaein, L Tassiulas, SLA-driven VM Scheduling in Mobile Edge Computing in *Proceedings of the IEEE International Conference on Cloud Computing*. pp. 750–757 (2016).
113. M Aazam, M St-Hilaire, C Lung, I Lambadaris, MeFoRE: QoE Based Resource Estimation at Fog to Enhance QoS in IoT in *Proceedings of the International Conference on Telecommunications*. pp. 1–5 (2016).
114. K Zhang, Y Mao, S Leng, S Maharjan, Y Zhang, Optimal Delay Constrained Offloading for Vehicular Edge Computing Networks in *Proceedings of the IEEE International Conference on Communications*. (IEEE), pp. 1–6 (2017).
115. O Skarlat, M Nardelli, S Schulte, S Dustdar, Towards QoS-aware Fog Service Placement in *Proceedings of the IEEE International Conference on Fog and Edge Computing*. (IEEE), pp. 89–96 (2017).
116. M Chen, et al., A Dynamic Service Migration Mechanism in Edge Cognitive Computing. *ACM Transactions on Internet Technol.* **19**, 1–15 (2019).
117. L Villas, A Boukerche, HD Oliveira, RD Araujo, A Loureiro, A Spatial Correlation Aware Algorithm to Perform Efficient Data Collection in Wireless Sensor Networks. *Ad Hoc Networks* **12**, 69 – 85 (2014).
118. Y Yao, Q Cao, A Vasilakos, EDAL: An Energy-Efficient, Delay-Aware, and Lifetime-Balancing Data Collection Protocol for Wireless Sensor Networks in *Proceedings of the IEEE International Conference on Mobile Ad-Hoc and Sensor Systems*. pp. 182–190 (2013).
119. Y Lin, B Kemme, M Patino-Martinez, R Jimenez-Peris, Enhancing Edge Computing with Database Replication in *Proceedings of the IEEE International Symposium on Reliable Distributed Systems*. (IEEE), pp. 45–54 (2007).
120. Z Chen, et al., Early Implementation Experience with Wearable Cognitive Assistance Applications in *Proceedings of the Workshop on Wearable Systems and Applications*. (ACM), pp. 33–38 (2015).
121. L Ma, S Yi, Q Li, Efficient Service Handoff Across Edge Servers via Docker Container Migration in *Proceedings of the ACM/IEEE Symposium on Edge Computing*. pp. 1–13 (2017).
122. D Roy, D De, A Mukherjee, R Buyya, Application-aware Cloudlet Selection for Computation Offloading in Multi-cloudlet Environment. *The J. Supercomput.*, 1–19 (2016).
123. X Chen, Q Shi, L Yang, J Xu, ThriftyEdge: Resource-efficient Edge Computing for Intelligent IoT Applications. *IEEE Netw.* **32**, 61–65 (2018).
124. X Chen, L Jiao, W Li, X Fu, Efficient Multi-user Computation Offloading for Mobile-edge Cloud Computing. *IEEE/ACM Transactions on Netw.* **24**, 2795–2808 (2015).
125. J Zhao, Q Li, Y Gong, K Zhang, Computation Offloading and Resource Allocation for Cloud Assisted Mobile Edge Computing in Vehicular Networks. *IEEE Transactions on Veh. Technol.* **68**, 7944–7956 (2019).
126. Y Xiao, M Krunz, QoE and Power Efficiency Tradeoff for Fog Computing Networks with Fog Node Cooperation in *Proceedings of the IEEE Conference on Computer Communications*. (IEEE), pp. 1–9 (2017).
127. M Satyanarayanan, P Bahl, R Caceres, N Davies, The Case for VM-based Cloudlets in Mobile Computing. *IEEE Pervasive Comput.* **8**, 14–23 (2009).
128. W Felter, A Ferreira, R Rajamony, J Rubio, An Updated Performance Comparison of Virtual Machines and Linux Containers in *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software*. pp. 171–172 (2015).
129. B Ismail, et al., Evaluation of Docker as Edge Computing Platform in *Proceedings of the IEEE Conference on Open Systems*. (IEEE), pp. 130–135 (2015).
130. A Madhavapeddy, et al., Unikernels: Library Operating Systems for the Cloud. *ACM SIGARCH Comput. Archit. News* **41**, 461–472 (2013).
131. R Morabito, V Cozzolino, A Ding, N Beijar, J Ott, Consolidate IoT Edge Computing with Lightweight Virtualization. *IEEE Netw.* **32**, 102–111 (2018).
132. A Burns, AJ Wellings, *Real-time Systems and Programming Languages: Ada 95, Real-time Java, and Real-time POSIX*. (Pearson Education), (2001).
133. J Wang, et al., Towards Scalable Edge-native Applications in *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing*. pp. 152–165 (2019).
134. Y Kang, et al., Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems*. p. 615–629 (2017).
135. BP Rimal, M Maier, M Satyanarayanan, Experimental Testbed for Edge Computing in Fiber-Wireless Broadband Access Networks. *IEEE Commun. Mag.* **56**, 160–167 (2018).
136. R Muñoz, et al., The ADRENALINE Testbed: An SDN/NFV Packet/Optical Transport Network and Edge/Core Cloud Platform for End-to-end 5G and IoT Services in *Proceedings of the European Conference on Networks and Communications*. pp. 1–5 (2017).
137. FP Tso, DR White, S Jouet, J Singer, DP Pezaros, The Glasgow Raspberry Pi Cloud: A Scale Model for Cloud Computing Infrastructures in *Proceedings of the First International Workshop on Resource Management of Cloud Computing*. pp. 108–112 (2013).
138. J Hasenburg, M Grambow, E Grunewald, S Huk, D Bermbach, MockFog: Emulating Fog Computing Infrastructure in the Cloud in *Proceedings of the First IEEE International Conference on Fog Computing*. (2019).
139. H Gupta, AV Dastjerdi, SK Ghosh, R Buyya, iFogSim: A Toolkit for Modeling and Simulation of Resource Management Techniques in the Internet of Things, Edge and Fog Computing Environments. *Software: Pract. Exp.* **47**, 1275–1296 (2017).
140. J Hasenburg, S Werner, D Bermbach, FogExplorer in *Proceedings of the 19th International Middleware Conference, Demos, and Posters (MIDDLEWARE 2018)*. (ACM), (2018).
141. J Hasenburg, S Werner, D Bermbach, Supporting the evaluation of fog-based IoT applications during the design phase in *Proceedings of the 5th Workshop on Middleware and Applications for the Internet of Things (M4IoT 2018)*. (ACM), (2018).
142. MM Lopes, WA Higashino, MAM Capretz, LF Bittencourt, MyiFogSim: A Simulator for Virtual Machine Migration in Fog Computing in *Companion Proceedings of the 10th International Conference on Utility and Cloud Computing*. pp. 47–52 (2017).
143. O Kolosov, G Yadgar, S Maheshwari, E Soljanin, Benchmarking in The Dark: On the Absence of Comprehensive Edge Datasets in *3rd USENIX Workshop on Hot Topics in Edge Computing*. (2020).
144. D Bermbach, Ph.D. thesis (Karlsruhe Institute of Technology) (2014).
145. T Rabl, et al., Solving Big Data Challenges for Enterprise Application Performance Management. *Proc. VLDB Endow.* **5** (2012).
146. C Binnig, D Kossmann, T Kraska, S Loesing, How is the Weather Tomorrow? Towards a Benchmark for the Cloud in *Proceedings of the 2nd International Workshop on Testing Database Systems*. pp. 1–6 (2009).
147. S Islam, K Lee, A Fekete, A Liu, How a Consumer Can Measure Elasticity for Cloud Platforms in *Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering*. p. 85–96 (2012).
148. J Kuhlenkamp, S Werner, MC Borges, D Ernst, D Wenzel, Benchmarking Elasticity of FaaS Platforms as a Foundation for Objective-driven Design of Serverless Applications in *Proceedings of The 35th ACM/SIGAPP Symposium on Applied Computing*. (2020).
149. D Bermbach, E Wittern, Benchmarking Web API Quality in *Proceedings of the 16th International Conference on Web Engineering*. pp. 188–206 (2016).
150. D Bermbach, E Wittern, Benchmarking web api quality – revisited. *J. Web Eng.* (2020).
151. L Shao, et al., User-perceived Service Availability: A Metric and an Estimation Approach in *Proceedings of the IEEE International Conference on Web Services*. pp. 647–654 (2009).
152. M Toeroe, F Tam, *Service Availability: Principles and Practice*. (John Wiley & Sons), (2012).
153. T Hauer, P Hoffmann, J Lunney, D Ardelean, A Diwan, Meaningful Availability in *17th USENIX Symposium on Networked Systems Design and Implementation*. pp. 545–557 (2020).
154. A Fox, EA Brewer, Harvest, Yield, and Scalable Tolerant Systems in *Proceedings of the 7th Workshop on Hot Topics in Operating Systems*. pp. 174–178 (1999).
155. M Klems, M Menzel, R Fischer, Consistency Benchmarking: Evaluating the Consistency Behavior of Middleware Services in the Cloud in *Service-Oriented Computing*, Lecture Notes in Computer Science, eds. P Maglio, M Weske, J Yang, M Fantinato. (Springer Berlin Hei-

delberg) Vol. 6470, pp. 627–634 (2010).
156. K Zellag, B Kemme, How Consistent is Your Cloud Application? in *Proceedings of the 3rd Symposium on Cloud Computing*. pp. 6:1–6:14 (2012).
157. H Wada, A Fekete, L Zhao, K Lee, A Liu, Data Consistency Properties and the Trade-offs in Commercial Cloud Storages: the Consumers' Perspective in *Proceedings of the 5th Conference on Innovative Data Systems Research*. pp. 134–143 (2011).
158. W Golab, X Li, MA Shah, Analyzing Consistency Properties for Fun and Profit in *Proceedings of the 30th Symposium on Principles of Distributed Computing*. pp. 197–206 (2011).
159. E Anderson, X Li, MA Shah, J Tucek, JJ Wylie, What Consistency Does Your Key-value Store Actually Provide? in *Proceedings of the 6th Workshop on Hot Topics in System Dependability*. pp. 1–16 (2010).
160. D Bermbach, J Kuhlenkamp, Consistency in Distributed Storage Systems: An Overview of Models, Metrics and Measurement Approaches in *Networked Systems*, Lecture Notes in Computer Science, eds. V Gramoli, R Guerraoui. (Springer Berlin Heidelberg) Vol. 7853, pp. 175–189 (2013).
161. F Pallas, D Bermbach, S Müller, S Tai, Evidence-Based Security Configurations for Cloud Datastores in *Proceedings of the the 32nd ACM Symposium on Applied Computing*. (2017).
162. F Pallas, J Günther, D Bermbach, Pick your Choice in HBase: Security or Performance in *Proceedings of the IEEE International Conference on Big Data*. (2017).
163. D Bermbach, E Wittern, Benchmarking Web API Quality-Revisited. *arXiv preprint arXiv:1903.07712* (2019).
164. G Apostolopoulos, V Peris, D Saha, Transport Layer Security: How Much Does it Really Cost? in *Proceedings of the 18th Annual Joint Conference of the IEEE Computer and Communications Societies*. Vol. 2, pp. 717–725 (1999).
165. K Kant, R Iyer, P Mohapatra, Architectural Impact of Secure Socket Layer on Internet Servers in *Proceedings of the International Conference on Computer Design*. pp. 7–14 (2000).
166. C Coarfa, P Druschel, DS Wallach, Performance Analysis of TLS Web Servers. *ACM Transactions on Comput. Syst.* **24**, 39–69 (2006).
167. S Shastri, V Banakar, M Wasserman, A Kumar, V Chidambaram, Understanding and Benchmarking the Impact of GDPR on Database Systems. *arXiv preprint arXiv:1910.00728* (2019).